

Software Development Roadmap

Today 1:49 AM

Module 4: Database Systems

What is a Database?

DBMS

RDBMS

ER Diagram

Keys

Normalization

SQL

DDL

DML

DCL

TCL

Joins

Views

Stored Procedures

Triggers

Indexing

Transactions

Practice with SQL Server and learn the basics of Oracle.

Module 5: HTML

HTML Structure

Tags

Forms

Tables

Images

Lists

Semantic HTML

Audio

Video

Module 6: CSS

Selectors

Colors

Fonts

Box Model

Position

Flexbox

Grid

Animations

Responsive Design

Module 7: Bootstrap

Grid System

Navbar

Cards

Forms

Buttons

- Modals
- Carousel
- Responsive Utilities
- Module 8: JavaScript
 - Variables
 - Functions
 - Scope
 - Objects
 - Arrays
 - DOM
 - Events
 - Async Programming
 - Promises
 - Fetch API
 - ES6 Features
- Module 9: jQuery
 - Selectors
 - Events
 - Effects
 - AJAX
 - DOM Manipulation
 - Form Validation
- Module 10: C#
 - Syntax
 - Variables
 - Loops
 - OOP
 - Exception Handling
 - Collections
 - LINQ
 - Delegates
 - Events
 - Generics
 - File Handling
- Module 11: .NET & ASP.NET Core
 - .NET Architecture
 - CLR
 - CTS
 - ASP.NET Core
 - MVC
 - Razor
 - Routing
 - Dependency Injection
 - Middleware
 - Web API
 - Authentication
 - Authorization
 - JWT
 - Entity Framework Core
 - CRUD
- Module 12: Web APIs

- REST
- HTTP Methods
- JSON
- API Testing
- Postman
- Swagger
- Module 13: Entity Framework Core
- Models
- Migrations
- Relationships
- CRUD
- LINQ Queries
- Module 14: Git & GitHub
- Git Basics
- Branches
- Merge
- Pull Requests
- Conflicts
- GitHub Workflow
- Module 15: Software Engineering
- SDLC
- Agile
- Scrum
- Requirement Gathering
- UML
- Documentation
- Unit Testing
- Module 16: Mobile Development
- Android Basics
- Activities
- Intents
- Layouts
- RecyclerView
- SQLite
- Firebase
- Module 17: AI & Machine Learning Basics
- AI
- ML
- Deep Learning
- Classification
- Regression
- CNN
- Transfer Learning
- TensorFlow
- Module 18: Enterprise Development
- 3-Tier Architecture
- SOLID Principles
- Design Patterns
- Repository Pattern
- Dependency Injection
- Module 19: Projects

You'll build:

Calculator
 Student Management System
 Employee Management System
 Library Management System
 Authentication System
 Inventory Management System
 Full Stack CRUD Application
 Module 20: Placement Preparation
 Aptitude
 Logical Reasoning
 Verbal Ability
 DSA Interview Questions
 DBMS Interview Questions
 OOP Interview Questions
 SQL Interview Questions
 HTML/CSS/JS Interview Questions
 .NET Interview Questions
 HR Interview
 Mock Interviews

How we'll learn

For every topic, I'll cover:

Theory from the basics.
 Real-world examples and analogies.
 Code examples.
 Hands-on exercises.
 Common interview questions.
 Practice problems.
 Mini project (where applicable).
 Summary and revision notes.

We'll treat this like a complete software development course, progressing step by step. We won't skip ahead until you're comfortable with the current topic. This way, you'll build a solid foundation and be well prepared for Hansa's Software Development role and future interviews.

Show moreShow less

Great. We will follow this as a **complete Software Development Course roadmap** and proceed **step-by-step from Module 4 onwards**.

Our learning pattern for every topic:

1. **Concept & Theory**
2. **Real-world analogy**
3. **Architecture / Internal working**
4. **Syntax & Code examples**
5. **Hands-on exercises**
6. **Interview questions**
7. **Common mistakes**

8. Revision notes

We will start with:

Module 4: Database Systems

Topic 1: What is a Database?

1. Introduction to Database

A **database** is an organized collection of data that can be easily stored, accessed, managed, and updated.

In simple words:

A database is a digital storage system where information is stored in a structured way so that applications can retrieve and manipulate it efficiently.

Real-world Example

Imagine a college.

It has information about:

- Students
- Courses
- Marks
- Attendance
- Teachers

Instead of storing everything in paper files, we store it electronically.

Example:

Student Database

Student_ID	Name	Branch	CGPA
101	Rahul	CSE	8.5
102	Priya	ECE	9.1
103	Arun	IT	8.8

This collection of related information is a database.

2. Why Do We Need Databases?

Before databases, organizations used:

File System

Example:

College Folder

```

| --- Students.txt
| --- Marks.txt
| --- Attendance.txt

```

Problems:

1. Data Redundancy

Same data stored multiple times.

Example:

Student name repeated in:

```

Students.txt
Marks.txt
Attendance.txt

```

This wastes storage.

2. Data Inconsistency

Different files may have different values.

Example:

Students.txt

Name: Kamesh

Marks.txt

Name: Kameshwar

Which one is correct?

3. Difficult Data Access

Finding information manually is slow.

Example:

"Find all students with CGPA > 9"

In file systems, we need to manually search.

4. Security Problems

Anyone can access files.

Database provides:

- Authentication
 - Authorization
 - Access control
-

5. No Concurrent Access

Multiple users cannot easily work together.

Example:

Bank system:

- Customer withdraws money
- Another transaction happens simultaneously

Database handles this safely.

3. Characteristics of Database

A good database provides:

1. Data Persistence

Data remains stored permanently.

Example:

After closing Amazon app:

Your orders are still available.

2. Data Integrity

Data should remain accurate.

Example:

Age cannot be:

```
Age = -5
```

Database prevents invalid data.

3. Data Security

Only authorized users can access data.

Example:

Bank employees cannot see customer passwords.

4. Data Sharing

Multiple users can access the same data.

Example:

In a hospital:

Doctors, nurses, billing department access patient data.

5. Backup and Recovery

Database can restore lost data.

Example:

System crash → Recover previous state.

4. Types of Databases

1. Hierarchical Database

Data is stored like a tree structure.

Example:

Company

|
|--- **Department**

|
|--- **Employee**

Used in old systems.

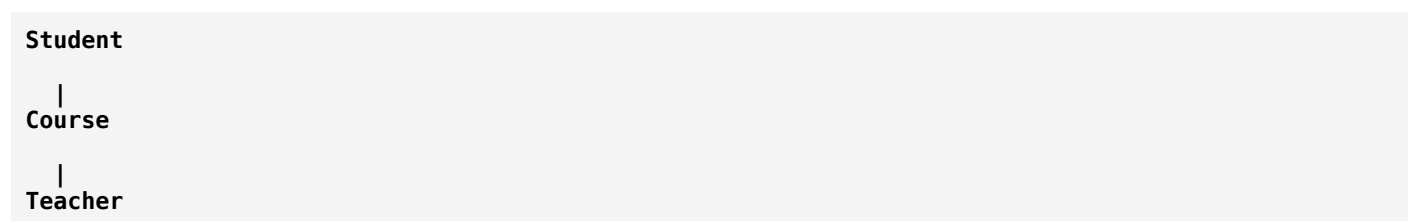
Example:

IBM IMS

2. Network Database

Data connected using graphs.

Example:



Supports many-to-many relationships.

3. Relational Database (Most Important)

Data stored in tables.

Example:

Student Table:

ID	Name	Branch
1	Amit	CSE
2	Ravi	ECE

Used by:

- MySQL
- Oracle
- SQL Server
- PostgreSQL

4. NoSQL Database

Stores unstructured data.

Examples:

- MongoDB
- Firebase
- Cassandra

Example:

JSON document:

```
JSON
{
  "name": "Kamesh",
  "skills": [
    "Java",
    "React",
    "Python"
  ]
}
```

5. Database Terminology

Table

Collection of related data.

Example:

Student table:

Student

ID | Name | Age

Row / Record

Single entry.

Example:

101 | Rahul | 20

Column / Attribute

Property of data.

Example:

**Name
Age
Branch**

Schema

Structure/design of database.

Example:

```
Student(  
  ID INT,  
  Name VARCHAR,
```

Age INT
)

Instance

Actual data stored at a particular time.

Example:

Today:

101 Rahul
102 Priya

Tomorrow:

101 Rahul
102 Priya
103 Arun

Instance changes.

Schema usually remains same.

6. Database Examples in Real Life

Banking System

Database stores:

- Customers
- Accounts
- Transactions
- Loans

E-Commerce

Amazon database stores:

- Products
- Users
- Orders
- Payments
- Reviews

Social Media

Instagram database stores:

- Users
- Posts
- Comments
- Likes
- Followers

7. Database vs File System

Feature	File System	Database
Storage	Files	Tables
Redundancy	High	Low
Security	Poor	High
Data Sharing	Difficult	Easy
Backup	Manual	Automatic
Searching	Slow	Fast
Multi-user	Limited	Supported

Interview Questions

Q1. What is a database?

Answer:

A database is an organized collection of related data stored electronically that allows efficient storage, retrieval, and management of information.

Q2. Why do we use databases?

Answer:

Databases are used to:

- Reduce redundancy
- Maintain consistency
- Provide security
- Support multiple users
- Enable fast data retrieval

Q3. Difference between data and database?

Data	Database
Raw facts	Organized collection of data
Example: Rahul, 20	Student table containing Rahul's details

Q4. What is a schema?

Schema defines the structure of a database including:

- Tables
- Columns
- Relationships
- Constraints

Practice Exercise

Answer these:

1. Design a database for a **Library Management System**.

Identify:

- Tables
- Columns
- Primary keys

2. Design database for:

Online Shopping Application

Find:

- Entities
- Relationships

Quick Revision Notes

Database:
Collection of organized data

Purpose:
Store
Retrieve
Manage
Secure data

Types:
Hierarchical
Network
Relational
NoSQL

Important Terms:
Table
Row
Column
Schema
Instance

Advantages:
Security
Integrity
Backup
Sharing

Next Topic:

DBMS (Database Management System)

We will cover:

- What is DBMS?
- DBMS architecture
- Components of DBMS
- DBMS advantages
- DBMS vs File System
- DBMS examples
- Interview questions
- SQL Server and Oracle introduction

Proceeding step-by-step.

Ok next

Module 4: Database Systems

Topic 2: DBMS (Database Management System)

1. What is DBMS?

DBMS (Database Management System) is software that allows users to **create, store, manage, retrieve, and manipulate data** in a database.

In simple words:

DBMS is a software layer between the user/application and the database that controls how data is stored and accessed.

Real-world Analogy

Think of a **library**.

Database:

Books stored in the library.

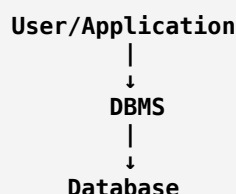
DBMS:

The librarian.

The librarian:

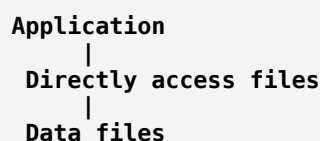
- Knows where every book is stored
- Gives books to users
- Maintains records
- Prevents unauthorized access

Similarly:



2. Why Do We Need DBMS?

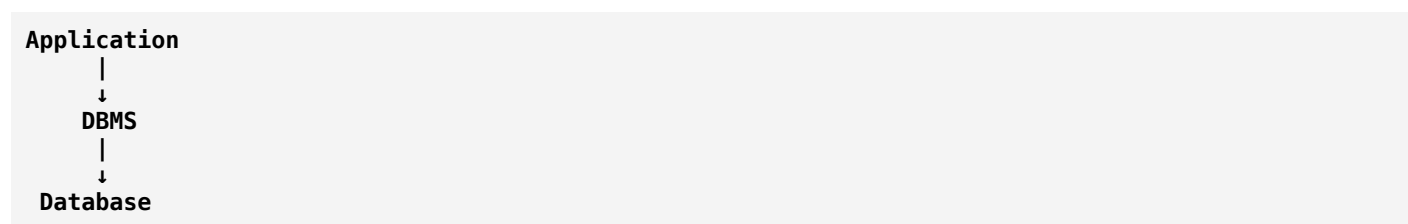
Without DBMS:



Problems:

- Data duplication
- Security issues
- Difficult searching
- No backup mechanism
- Data inconsistency

With DBMS:



DBMS handles everything.

3. Functions of DBMS

A DBMS performs many important operations.

1. Data Storage

DBMS stores data efficiently.

Example:

Student information:

Student Table		
ID	Name	Branch
101	Ravi	CSE
102	Priya	ECE

2. Data Retrieval

Allows users to search data.

Example:

Find student with ID 101:

```

SQL
SELECT *
FROM Student
WHERE ID = 101;
  
```

Output:

```

101 Ravi CSE
  
```

3. Data Manipulation

DBMS allows:

Insert

Adding new data:

```
SQL
INSERT INTO Student
VALUES(103, 'Arun', 'IT');
```

Update

Changing existing data:

```
SQL
UPDATE Student
SET Branch='CSE'
WHERE ID=103;
```

Delete

Removing data:

```
SQL
DELETE FROM Student
WHERE ID=103;
```

4. Data Security

DBMS controls access.

Example:

Bank database:

Admin:

Can view all data

Customer:

Can view only own account

5. Data Integrity

Maintains accuracy of data.

Example:

Student age:

Allowed:

Age = 20

Not allowed:

Age = -10

6. Backup and Recovery

DBMS automatically maintains backups.

Example:

Server crash:

Before:

Account Balance = ₹50,000

After recovery:

Account Balance = ₹50,000

7. Concurrency Control

Allows multiple users to access data simultaneously.

Example:

Bank:

User A:

Withdraw ₹5000

User B:

Check balance

DBMS ensures correct results.

4. Components of DBMS

A DBMS consists of five major components.

1. Hardware

Physical devices:

- Servers
- Hard disks
- Network devices

Example:

Database server machine.

2. Software

DBMS software:

Examples:

- Oracle
 - MySQL
 - SQL Server
 - PostgreSQL
-

3. Data

Actual stored information.

Example:

```
Employee Database
ID | Name | Salary
```

4. Procedures

Rules and instructions.

Example:

- Backup procedure
 - Security policies
 - Data recovery steps
-

5. Database Users

People who interact with DBMS.

Types:

Database Administrator (DBA)

Responsible for:

- Database security
- Backup
- Performance tuning
- User management

Developers

Write applications using databases.

Example:

Backend developers.

End Users

Normal users.

Example:

Customers using banking apps.

5. DBMS Architecture

DBMS follows a three-level architecture.

1. External Level (View Level)

User's view of data.

Example:

Bank customer sees:

Account Number
Balance
Transactions

They don't see internal database details.

2. Conceptual Level (Logical Level)

Defines database structure.

Contains:

- Tables
- Relationships
- Constraints

Example:

Customer
Account

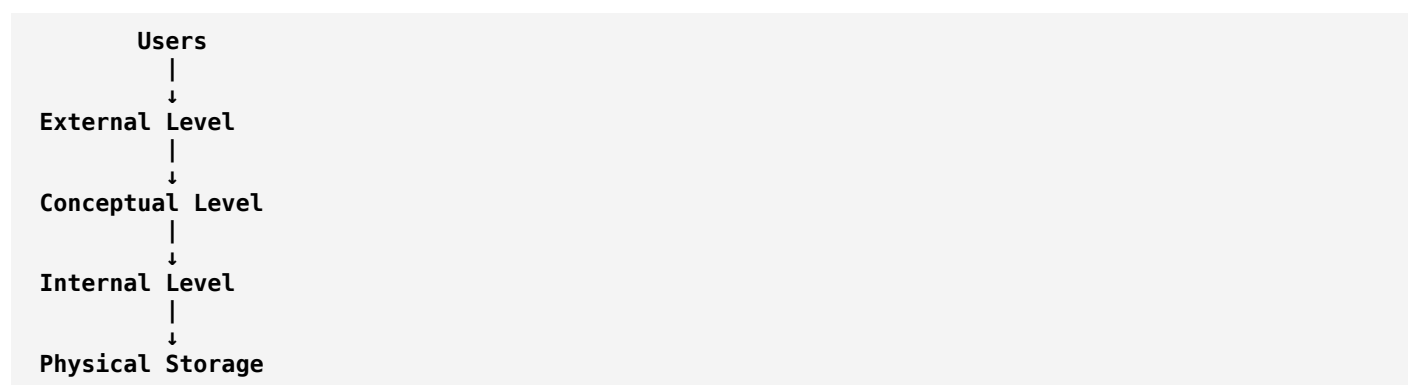
3. Internal Level (Physical Level)

Describes how data is actually stored.

Example:

- File organization
- Indexing
- Storage blocks

Architecture:

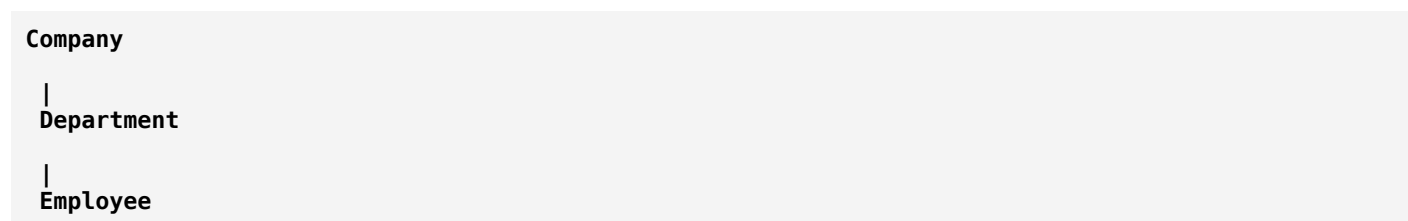


6. Types of DBMS

1. Hierarchical DBMS

Data arranged as tree structure.

Example:



2. Network DBMS

Uses graph structure.

Supports many-to-many relationships.

3. Relational DBMS (RDBMS)

Most commonly used.

Data stored in tables.

Examples:

- MySQL
- Oracle
- SQL Server
- PostgreSQL

4. Object-Oriented DBMS

Stores data as objects.

Used in:

- Engineering applications
- CAD systems

7. Popular DBMS Software

DBMS	Company	Usage
Oracle DB	Oracle	Enterprise applications
MySQL	Oracle	Web applications
SQL Server	Microsoft	Enterprise + .NET applications
PostgreSQL	Open Source	Advanced applications
MongoDB	MongoDB Inc.	NoSQL applications

8. DBMS Advantages

Advantage	Explanation
Reduced redundancy	Avoids duplicate data
Security	Controls access
Data sharing	Multiple users
Integrity	Maintains accuracy
Backup	Data recovery
Fast searching	Query processing

9. DBMS Disadvantages

Problem	Explanation
Cost	Software and hardware cost
Complexity	Requires skilled administrators
Large size	Requires more storage
Maintenance	Regular updates needed

DBMS vs File System

Feature	File System	DBMS
Data storage	Files	Database
Redundancy	High	Low
Security	Low	High
Backup	Manual	Automatic
Querying	Difficult	Easy using SQL
Multiple users	Poor	Excellent
Data integrity	Low	High

DBMS Interview Questions

Q1. What is DBMS?

Answer:

DBMS is software that manages databases by allowing users to store, retrieve, update, and control data efficiently.

Q2. What are the advantages of DBMS?

Answer:

- Data security
- Data integrity
- Reduced redundancy
- Backup and recovery
- Concurrent access

Q3. Who is DBA?

Answer:

A Database Administrator is responsible for managing database security, performance, backup, and maintenance.

Q4. What are the three levels of DBMS architecture?

Answer:

1. External Level
 2. Conceptual Level
 3. Internal Level
-

Q5. Difference between DBMS and RDBMS?

DBMS:

- General database management system
- May not use tables

RDBMS:

- Data stored in related tables
 - Uses keys and relationships
 - Uses SQL
-

Quick Revision Notes

DBMS:

Software to manage databases.

Functions:

- Store data
- Retrieve data
- Update data
- Security
- Backup
- Concurrency control

Components:

1. Hardware
2. Software
3. Data
4. Procedures
5. Users

Architecture:

External

Conceptual
Internal

Users:
DBA
Developers
End Users

Next Topic:

RDBMS (Relational Database Management System)

We will cover:

- What is RDBMS?
- Tables, Rows, Columns
- Relationships
- Primary Key
- Foreign Key
- Constraints
- RDBMS vs DBMS
- MySQL, Oracle, SQL Server examples
- Interview questions

Proceeding step-by-step.

Ok next

Module 4: Database Systems

Topic 3: RDBMS (Relational Database Management System)

1. What is RDBMS?

RDBMS (Relational Database Management System) is a type of DBMS where data is stored in the form of **tables (relations)**, and relationships are created between these tables using keys.

In simple words:

RDBMS stores data in rows and columns and connects related information using relationships.

Real-world Example

Consider an online shopping application.

Instead of storing everything in one huge table:

Customer Table

Customer_ID	Name	Email
101	Rahul	rahul@gmail.com
102	Priya	priya@gmail.com

Order Table

Order_ID	Customer_ID	Product
5001	101	Laptop
5002	102	Mobile

Here:

Customer_ID connects both tables.

This connection is called a **relationship**.

2. Why is it called Relational?

Because data is stored as **relations**.

In RDBMS:

- Table = Relation
- Row = Tuple
- Column = Attribute

Example:

Student Relation:

Roll_No	Name	Branch
1	Amit	CSE
2	Ravi	ECE

Here:

Relation = Student table

3. Structure of RDBMS

An RDBMS consists of:

Database

```

| --- Table 1
|       |
|       | Rows + Columns
| --- Table 2
|       |
|       | Rows + Columns
| --- Relationships

```

4. Important RDBMS Terminology

1. Table

A collection of related data arranged in rows and columns.

Example:

Employee Table:

Emp_ID	Name	Salary
1	John	50000
2	Alex	60000

2. Row (Tuple)

A single record in a table.

Example:

```
1 | John | 50000
```

3. Column (Attribute)

A property of an entity.

Example:

```

Name
Salary
Department

```

4. Domain

Set of valid values for an attribute.

Example:

Age column:

Domain = 1 to 100

Invalid:

Age = -5

5. Schema

The design/structure of a database.

Example:

```
Student(
  Roll_No INT,
  Name VARCHAR(50),
  Age INT
)
```

6. Instance

Actual data present at a particular time.

Example:

Today:

ID	Name
1	Rahul

Tomorrow:

ID	Name
1	Rahul
2	Priya

The instance changes.

5. Characteristics of RDBMS

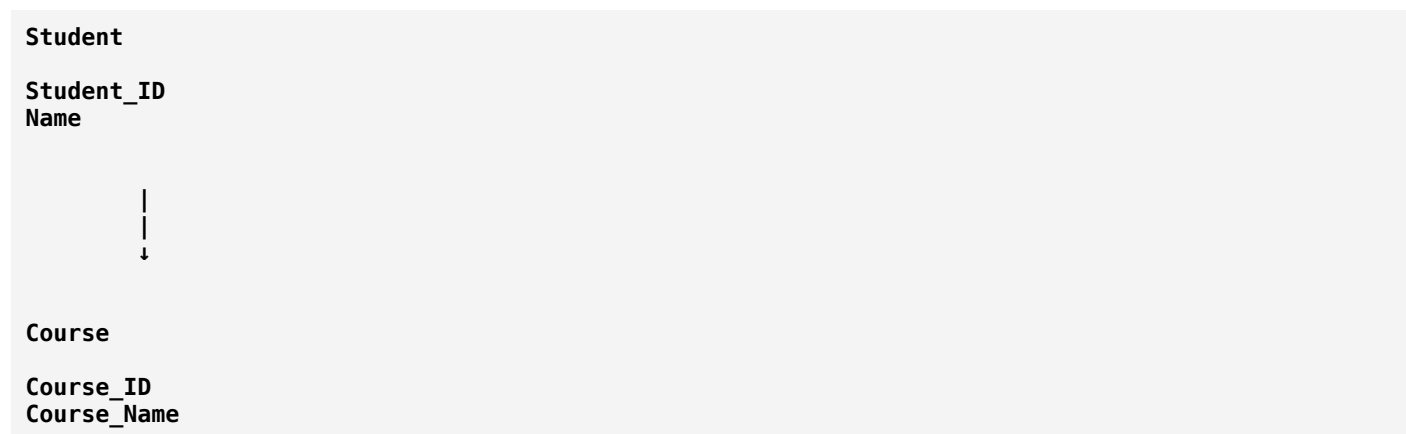
1. Data Stored in Tables

Example:

```
Employee
ID | Name | Salary
```

2. Relationships Between Tables

Example:



3. Uses SQL

RDBMS uses Structured Query Language.

Example:

Retrieve all students:

```

SQL
SELECT * FROM Student;
  
```

4. Supports Constraints

Rules that maintain data accuracy.

Examples:

- Primary Key
- Foreign Key
- NOT NULL
- UNIQUE
- CHECK

6. Keys in RDBMS

Keys are used to identify records and create relationships.

Primary Key

A column that uniquely identifies each row.

Example:

Student Table:

Roll_No	Name
101	Rahul
102	Priya

Roll_No is Primary Key.

Properties:

- Unique
- Cannot be NULL
- Only one primary key per table

Example:

```
SQL
CREATE TABLE Student(
Roll_No INT PRIMARY KEY,
Name VARCHAR(50)
);
```

Foreign Key

A column that refers to the primary key of another table.

Example:

Student Table:

Student_ID	Name
1	Amit

Marks Table:

Mark_ID	Student_ID	Marks
10	1	95

Student_ID in Marks table is a Foreign Key.

Candidate Key

A column that can become a primary key.

Example:

Employee:

ID	Email	Name
1	a@gmail.com	Raj

Both:

- ID
- Email

can uniquely identify employees.

They are candidate keys.

Super Key

Any attribute combination that uniquely identifies a row.

Example:

Student_ID
Student_ID + Name
Student_ID + Age

All are super keys.

Composite Key

Primary key made of multiple columns.

Example:

Student Course:

Student_ID	Course_ID
101	CSE01

Both together identify a record.

7. Relationships in RDBMS

1. One-to-One Relationship

One record relates to one record.

Example:

Person → Passport

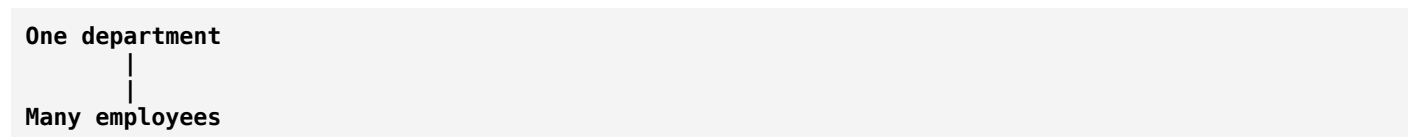
One person has one passport

2. One-to-Many Relationship

Most common.

Example:

Department → Employees

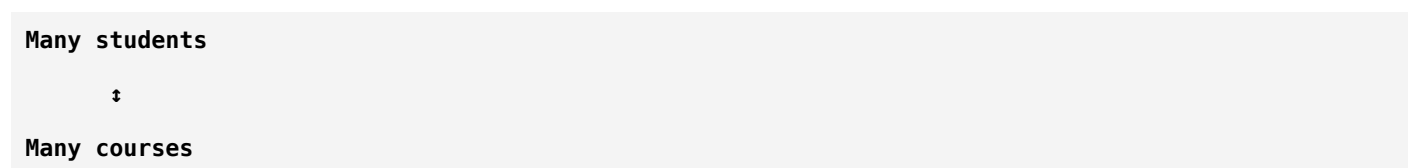


3. Many-to-Many Relationship

Many records relate to many records.

Example:

Students and Courses



Requires a junction table.

Example:

Student_Course:

Student_ID	Course_ID
1	CSE
2	ECE

8. RDBMS Examples

Oracle Database

Used in:

- Banking
- Large enterprises

Microsoft SQL Server

Used in:

- .NET applications
- Enterprise software

MySQL

Used in:

- Web applications
- Startups

PostgreSQL

Used in:

- Advanced applications
- Data analytics

9. DBMS vs RDBMS

DBMS	RDBMS
General database system	Relational database system
Data may not be stored in tables	Data stored in tables
Relationships may not exist	Supports relationships
Less secure	More secure
No strict constraints	Supports constraints
Example: File-based systems	Oracle, MySQL, SQL Server

10. Advantages of RDBMS

1. Data Integrity

Ensures accurate data.

2. Reduced Data Redundancy

Avoids duplicate storage.

3. Data Security

Provides:

- User authentication
- Permissions

4. Easy Data Retrieval

SQL makes searching easy.

Example:

```
SQL
SELECT Name
FROM Employee
WHERE Salary > 50000;
```

5. Scalability

Handles millions of records.

11. Limitations of RDBMS

Limitation	Explanation
Complex structure	Requires database design
Less suitable for unstructured data	Images/videos/social data
Scaling challenges	Very large distributed systems

Interview Questions

Q1. What is RDBMS?

Answer:

RDBMS is a database management system that stores data in tables and establishes relationships between tables using keys.

Q2. What is a relation?

A relation is a table containing rows and columns in an RDBMS.

Q3. Difference between Primary Key and Foreign Key?

Primary Key	Foreign Key
Uniquely identifies record	Creates relationship

Primary Key	Foreign Key
Cannot be NULL	Can contain NULL
One per table	Multiple allowed

Q4. What is normalization?

Normalization is the process of organizing data to reduce redundancy and improve consistency.

(We will study this in detail later.)

Quick Revision Notes

RDBMS:

Data → Tables

Table:

Rows + Columns

Important Terms:

Relation = Table

Tuple = Row

Attribute = Column

Keys:

Primary Key

Foreign Key

Candidate Key

Super Key

Composite Key

Relationships:

1:1

1:M

M:N

Examples:

Oracle

MySQL

SQL Server

PostgreSQL

Next Topic:

ER Diagram (Entity Relationship Diagram)

We will cover:

- Entities
- Attributes
- Relationships
- ER symbols
- Cardinality

- Weak entities
- Designing ER diagrams
- Real-world examples
- Interview questions

Proceeding step-by-step.

Ok next

Module 4: Database Systems

Topic 4: ER Diagram (Entity Relationship Diagram)

1. What is an ER Diagram?

ER Diagram (Entity Relationship Diagram) is a visual representation of the structure of a database.

It shows:

- **Entities** (objects)
- **Attributes** (properties)
- **Relationships** (connections)

In simple words:

An ER Diagram is a blueprint of a database that helps us design tables before implementation.

2. Why Do We Need ER Diagrams?

Before creating a database, we need to understand:

- What data should be stored?
- What are the relationships between data?
- How many tables are required?
- What keys are needed?

ER diagrams help database designers plan the database structure.

3. Components of ER Diagram

An ER diagram mainly contains:

1. Entity
2. Attribute
3. Relationship
4. Cardinality

1. Entity

An **entity** is a real-world object about which we store information.

Examples:

- Student
- Employee
- Customer
- Product
- Department

Representation:

```
+-----+
|  Student  |
+-----+
```

Entity becomes a table in RDBMS.

Example:

Entity:

Student

becomes:

Student_ID	Name	Branch
101	Rahul	CSE

Types of Entities

1. Strong Entity

An entity that has its own primary key.

Example:

Student

Student_ID → Primary Key

It can exist independently.

2. Weak Entity

An entity that depends on another entity for existence.

Example:

Employee and Dependent

Employee:

Emp_ID	Name
101	John

Dependent:

Dependent_Name	Age
Child	5

Dependent cannot exist without Employee.

Representation:

Double rectangle:



2. Attributes

Attributes describe properties of an entity.

Example:

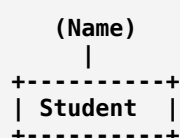
Student Entity:

Student

```
|
| --- Roll_No
| --- Name
| --- Age
| --- Branch
```

Representation:

Oval shape:



|
(Age)

Types of Attributes

1. Simple Attribute

Cannot be divided further.

Example:

```
Age
Salary
Gender
```

2. Composite Attribute

Can be divided into smaller parts.

Example:

Name:

```
Name
|
|-- First Name
|
|-- Last Name
```

Address:

```
Address
|
|-- Street
|-- City
|-- State
```

3. Single-Valued Attribute

Only one value.

Example:

```
Date_of_Birth
```

A person has one DOB.

4. Multi-Valued Attribute

Can have multiple values.

Example:

Student phone numbers:

Phone_Number
9876543210
9123456789

Representation:

Double oval.

5. Derived Attribute

Calculated from another attribute.

Example:

Age derived from DOB.

DOB → Age

Representation:

Dashed oval.

3. Relationship

A relationship shows association between entities.

Example:

Student enrolls in Course.

Student	-----	Enrolls	-----	Course
---------	-------	---------	-------	--------

Representation:

Diamond shape:

<Enrolls>		
Student		Course

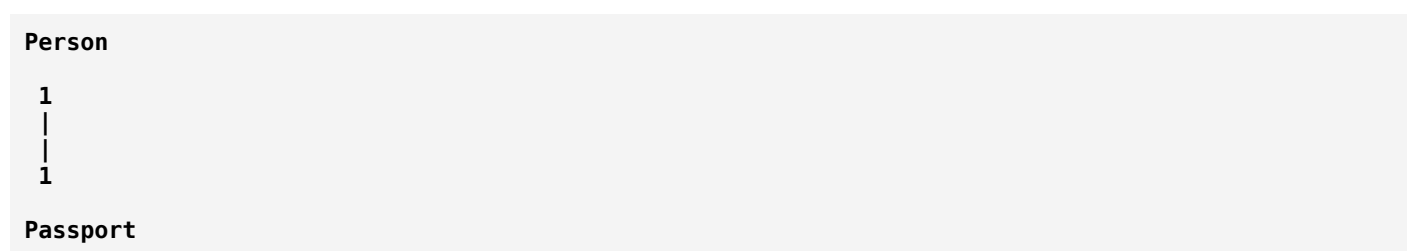
Types of Relationships

1. One-to-One (1:1)

One entity is related to one entity.

Example:

Person and Passport



Meaning:

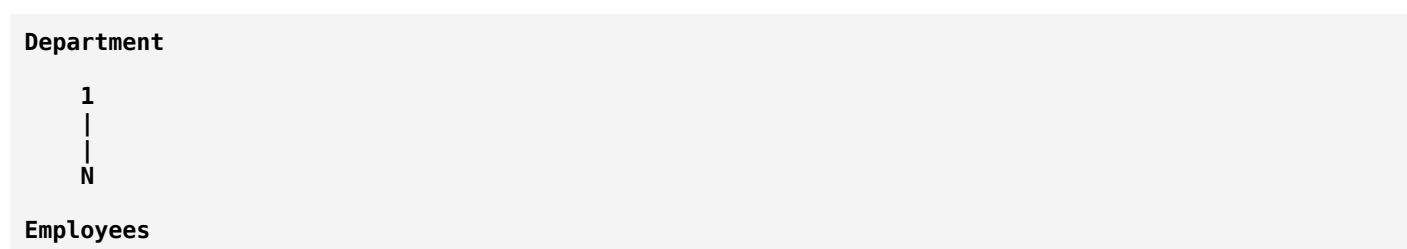
One person has one passport.

2. One-to-Many (1:N)

One entity relates to many entities.

Example:

Department and Employees



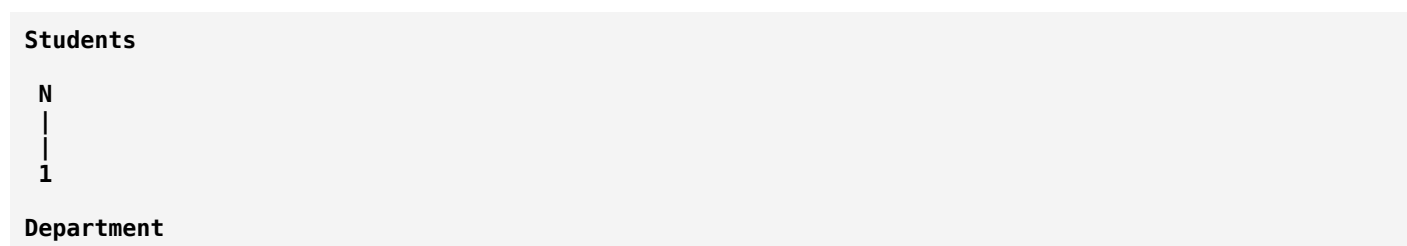
One department has many employees.

3. Many-to-One (N:1)

Many entities relate to one entity.

Example:

Many students belong to one department.

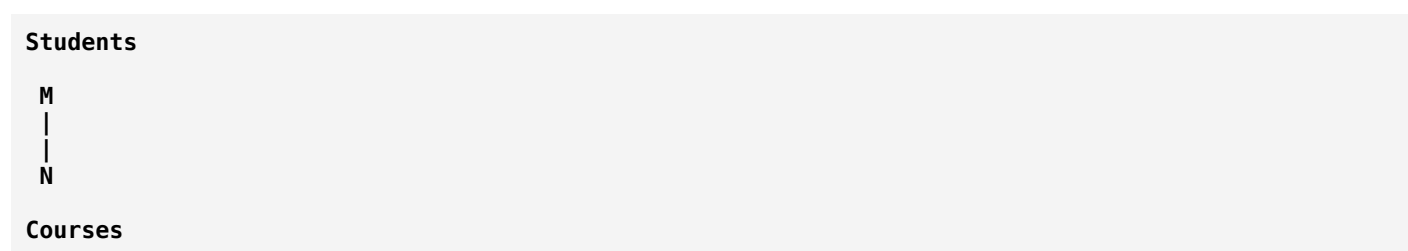


4. Many-to-Many (M:N)

Many entities relate to many entities.

Example:

Students and Courses



Example:

A student can take many courses.

A course can have many students.

4. Cardinality

Cardinality defines the number of relationships between entities.

Types:

One-to-One

1 : 1

Example:

Employee → ID Card

One-to-Many

1 : N

Example:

Company → Employees

Many-to-Many

M : N

Example:

Students → Subjects

5. ER Diagram Symbols

Component	Symbol
Entity	Rectangle
Weak Entity	Double Rectangle
Attribute	Oval
Multi-valued Attribute	Double Oval
Relationship	Diamond
Key Attribute	Underlined
Derived Attribute	Dashed Oval

6. Example: College Database ER Diagram

Entities:

Student

Attributes:

Student_ID
Name
Age
Branch

Course

Attributes:

Course_ID
Course_Name
Credits

Relationship:

Student
|
Enrolls
|
Course

Relationship:

Many-to-Many

Because:

- One student can enroll in many courses

- One course can have many students

7. Converting ER Diagram to Tables

Rules:

Entity → Table

ER Entity:

Student

Table:

Student(Student_ID, Name, Age)

Attribute → Column

Example:

Name
Age

becomes:

Columns

Primary Key → Primary Key Column

Example:

Student_ID

Relationship → Foreign Key

Example:

Department:

Department_ID

Employee:

Employee_ID
Department_ID

Department_ID becomes foreign key.

8. Advantages of ER Diagram

Advantage	Explanation
Easy visualization	Database structure is clear
Reduces errors	Find design problems early
Better communication	Developers and clients understand
Improves database design	Proper relationships

9. ER Diagram Interview Questions

Q1. What is an ER diagram?

Answer:

An ER diagram is a graphical representation of entities, attributes, and relationships used to design a database.

Q2. Difference between Entity and Attribute?

Entity	Attribute
Object in real world	Property of entity
Example: Student	Example: Name

Q3. What is a weak entity?

A weak entity does not have its own primary key and depends on another entity for existence.

Q4. What is cardinality?

Cardinality defines the number of instances of one entity that can be associated with another entity.

Examples:

- 1:1
- 1:N
- M:N

Quick Revision Notes

ER Diagram:

Purpose:

Database design blueprint

Components:

Entity

↓

Object

Attribute

↓

Property

Relationship

↓

Connection

Entity Types:

Strong Entity

Weak Entity

Attributes:

Simple

Composite

Single-valued

Multi-valued

Derived

Relationships:

1:1

1:N

M:N

Symbols:

Rectangle → Entity

Oval → Attribute

Diamond → Relationship

Next Topic:

Database Keys

We will cover:

- What are keys?
- Primary Key
- Foreign Key
- Candidate Key
- Super Key
- Alternate Key
- Composite Key
- Unique Key
- Surrogate Key

- Key differences
- Interview questions

Proceeding step-by-step.

Ok next

Module 4: Database Systems

Topic 5: Database Keys

1. What are Keys in Database?

A **key** is an attribute (column) or a combination of attributes used to:

- Uniquely identify records in a table
- Create relationships between tables
- Maintain data integrity

In simple words:

A key helps the database identify each row uniquely.

Real-world Example

Student Table:

Student_ID	Name	Email	Branch
101	Rahul	rahul@gmail.com	CSE
102	Priya	priya@gmail.com	ECE

Here:

- Student_ID uniquely identifies each student.
- Email can also uniquely identify a student.

These are called **keys**.

2. Types of Keys in DBMS

The major types of keys are:

1. Super Key

2. Candidate Key
3. Primary Key
4. Alternate Key
5. Foreign Key
6. Composite Key
7. Unique Key
8. Surrogate Key

1. Super Key

Definition

A **Super Key** is any set of one or more attributes that can uniquely identify a row in a table. It may contain extra attributes.

Example:

Employee Table:

Emp_ID	Email	Name
101	a@gmail.com	John

Possible Super Keys:

```
Emp_ID
Email
Emp_ID + Name
Email + Name
Emp_ID + Email
```

All can uniquely identify a record.

Important Point:

Every Candidate Key is a Super Key.

But every Super Key is not a Candidate Key.

2. Candidate Key

Definition

A **Candidate Key** is a minimal Super Key.

Meaning:

- It uniquely identifies records.
- It does not contain unnecessary attributes.

Example:

Employee Table:

Emp_ID	Email	Name
101	a@gmail.com	John

Candidate Keys:

Emp_ID

Email

Not:

Emp_ID + Name

because Name is unnecessary.

3. Primary Key

Definition

A **Primary Key** is the candidate key selected by the database designer to uniquely identify each record.

Example:

Student Table:

Roll_No	Name	Branch
101	Rahul	CSE
102	Priya	ECE

Primary Key:

Roll_No

Properties of Primary Key

1. Unique

Duplicate values are not allowed.

Invalid:

ID	Name
101	Rahul
101	Priya

2. Cannot be NULL

Invalid:

ID	Name
NULL	Rahul

3. Only One Primary Key per Table

Example:

```
SQL
CREATE TABLE Student(
Roll_No INT PRIMARY KEY,
Name VARCHAR(50)
);
```

4. Alternate Key

Definition

Candidate keys that are not selected as primary keys are called alternate keys.

Example:

Employee:

Emp_ID	Email	Name
101	john@gmail.com	John

Candidate Keys:

```
Emp_ID
Email
```

Selected Primary Key:

Emp_ID

Alternate Key:

Email

5. Foreign Key

Definition

A Foreign Key is an attribute that creates a relationship between two tables by referring to the primary key of another table.

Example:

Department Table

Dept_ID	Dept_Name
10	CSE
20	ECE

Primary Key:

Dept_ID

Student Table

Student_ID	Name	Dept_ID
101	Rahul	10
102	Priya	20

Here:

Student.Dept_ID

is a Foreign Key.

It references:

Department.Dept_ID

Properties of Foreign Key

- Can contain duplicate values
- Can contain NULL values

- Multiple foreign keys allowed in one table

6. Composite Key

Definition

A key created using two or more columns together is called a Composite Key.

Example:

Student_Course Table:

Student_ID	Course_ID	Grade
101	CSE01	A
101	ECE01	B

Neither column alone is unique.

But:

Student_ID + Course_ID

uniquely identifies a record.

7. Unique Key

Definition

A Unique Key ensures that all values in a column are different.

Example:

Employee:

ID	Email
1	a@gmail.com
2	b@gmail.com

Email can be a Unique Key.

Difference between Primary Key and Unique Key

Primary Key	Unique Key
Only one per table	Multiple allowed

Primary Key	Unique Key
Cannot contain NULL	Can contain NULL (usually one NULL allowed depending on DB)
Main identifier	Ensures uniqueness

8. Surrogate Key

Definition

A Surrogate Key is an artificial key created by the system.

It has no business meaning.

Example:

Instead of:

Email as ID

Create:

User_ID

Automatically generated:

User_ID	Email
1	abc@gmail.com
2	xyz@gmail.com

Used in:

- Large systems
- E-commerce
- Banking applications

9. Primary Key vs Foreign Key

Primary Key	Foreign Key
Uniquely identifies rows	Creates relationship
Only one per table	Multiple allowed
Cannot be NULL	May be NULL
No duplicate values	Duplicates allowed

Primary Key	Foreign Key
Defined in same table	References another table

10. Candidate Key vs Primary Key

Candidate Key	Primary Key
Possible keys	Selected key
Multiple possible	Only one
May become primary	Actual identifier

11. Keys Example: Complete Database Design

Customer Table

Customer_ID	Name	Email
1	Rahul	r@gmail.com
2	Priya	p@gmail.com

Keys:

Primary Key:
Customer_ID

Candidate Keys:
Customer_ID, Email

Alternate Key:
Email

Orders Table

Order_ID	Customer_ID	Amount
5001	1	2000
5002	2	3000

Keys:

Primary Key:
Order_ID

Foreign Key:
Customer_ID

12. SQL Example

Creating Primary Key:

```
SQL
CREATE TABLE Student(
Student_ID INT PRIMARY KEY,
Name VARCHAR(50)
);
```

Creating Foreign Key:

```
SQL
CREATE TABLE Orders(
Order_ID INT PRIMARY KEY,
Customer_ID INT,

FOREIGN KEY(Customer_ID)
REFERENCES Customer(Customer_ID)
);
```

Interview Questions

Q1. What is a primary key?

Answer:

A primary key is a column or set of columns that uniquely identifies each record in a table and cannot contain NULL values.

Q2. Can a table have multiple primary keys?

Answer:

No. A table can have only one primary key.

Q3. Can a foreign key contain duplicate values?

Answer:

Yes. Foreign keys can contain duplicate values because many records can refer to the same parent record.

Q4. Difference between Super Key and Candidate Key?

Super Key	Candidate Key
May contain unnecessary attributes	Minimal key
More possibilities	Only efficient keys

Quick Revision Notes

Keys:

Super Key

↓

Candidate Key

↓

Primary Key

Primary Key:

- Unique
- NOT NULL
- One per table

Foreign Key:

- Connects tables
- References primary key

Composite Key:

- Multiple columns

Alternate Key:

- Candidate key not selected

Surrogate Key:

- Artificial/system generated

Next Topic:

Normalization

We will cover:

- Why normalization is needed
- Data anomalies
- Functional dependency
- 1NF
- 2NF
- 3NF
- BCNF
- Denormalization
- Real-world examples
- Interview questions

Proceeding step-by-step.

Ok next

Module 4: Database Systems

Topic 6: Database Normalization

1. What is Normalization?

Normalization is the process of organizing data in a database to:

- Reduce data redundancy (duplicate data)
- Improve data consistency
- Avoid data anomalies
- Improve database design

In simple words:

Normalization means dividing large tables into smaller related tables to store data efficiently.

2. Why Do We Need Normalization?

Consider this table:

Student_Course Table

Student_ID	Student_Name	Course	Instructor	Instructor_Phone
101	Rahul	DBMS	Dr. Kumar	9876543210
102	Priya	DBMS	Dr. Kumar	9876543210
103	Amit	Java	Dr. Rao	9123456780

Problems:

- Instructor information is repeated.
- Phone numbers are duplicated.
- Updating one value requires multiple changes.

Normalization solves these problems.

3. Problems Without Normalization (Data Anomalies)

These problems are called **anomalies**.

There are three major anomalies:

1. Insertion Anomaly

Problem while inserting new data.

Example:

Table:

Student	Course	Instructor
Rahul	DBMS	Kumar

Suppose a new instructor joins but has no students yet.

We cannot insert instructor information because Student details are required.

2. Update Anomaly

Problem while updating data.

Example:

Course	Instructor	Phone
DBMS	Kumar	1111
SQL	Kumar	1111

If Kumar changes phone number:

We need to update multiple rows.

If one row is missed:

Data becomes inconsistent.

3. Delete Anomaly

Problem while deleting data.

Example:

Student	Course	Instructor
Rahul	DBMS	Kumar

If Rahul leaves and we delete this row:

Instructor Kumar's information is also lost.

4. Functional Dependency

Functional dependency describes a relationship between attributes.

It means:

One attribute determines another attribute.

Notation:

$A \rightarrow B$

Means:

A determines B.

Example:

Student_ID $\rightarrow$ Student_Name

Because:

Given Student_ID, we can find Student_Name.

Table:

Student_ID	Name
101	Rahul
102	Priya

Knowing Student_ID gives Name.

5. Normal Forms

Normalization is divided into levels called **Normal Forms**.

Main normal forms:

1. First Normal Form (1NF)
2. Second Normal Form (2NF)
3. Third Normal Form (3NF)
4. Boyce-Codd Normal Form (BCNF)

1. First Normal Form (1NF)

Rule:

A table is in 1NF if:

1. Each column contains atomic values.
2. No repeating groups.
3. Each row is unique.

Example: Not in 1NF

Student	Phone Numbers
Rahul	9876,9123

Problem:

Multiple values in one column.

Convert to 1NF:

Student	Phone
Rahul	9876
Rahul	9123

Now each field has a single value.

2. Second Normal Form (2NF)

Conditions:

A table must:

1. Be in 1NF
2. Have no partial dependency

What is Partial Dependency?

When a non-key attribute depends on only part of a composite key.

Example:

Student_Course:

Student_ID	Course_ID	Student_Name	Course_Name
101	C1	Rahul	DBMS

Primary Key:

Student_ID + Course_ID

Problem:

Student_Name depends only on:

Student_ID

Course_Name depends only on:

Course_ID

This is partial dependency.

Convert to 2NF:

Student Table

Student_ID	Student_Name
101	Rahul

Course Table

Course_ID	Course_Name
C1	DBMS

Enrollment Table

Student_ID	Course_ID
101	C1

Now it is in 2NF.

3. Third Normal Form (3NF)

Conditions:

A table must:

1. Be in 2NF
2. Have no transitive dependency

What is Transitive Dependency?

When:

$A \rightarrow B$

and

$B \rightarrow C$

then:

$A \rightarrow C$

Example:

Employee Table:

Emp_ID	Dept_ID	Dept_Name
101	10	CSE

Dependencies:

$Emp_ID \rightarrow Dept_ID$
 $Dept_ID \rightarrow Dept_Name$

Therefore:

$Emp_ID \rightarrow Dept_Name$

This is transitive dependency.

Convert to 3NF:

Employee Table

Emp_ID	Dept_ID
101	10

Department Table

Dept_ID	Dept_Name
10	CSE

4. BCNF (Boyce-Codd Normal Form)

BCNF is a stronger version of 3NF.

Rule:

Every determinant must be a candidate key.

Example:

If:

Teacher → Subject

Teacher is not a candidate key.

Then BCNF violation occurs.

6. Normalization Example (Complete)

Before Normalization

Student Table:

ID	Name	Course	Teacher
1	Rahul	DBMS	Kumar
2	Priya	DBMS	Kumar

Problems:

- Teacher repeated
- Course repeated

After Normalization

Student

Student_ID	Name
1	Rahul
2	Priya

Course

Course_ID	Course_Name	Teacher
C1	DBMS	Kumar

Enrollment

Student_ID	Course_ID
1	C1
2	C1

7. Normalization Levels Summary

Normal Form	Removes
1NF	Repeating values
2NF	Partial dependency
3NF	Transitive dependency
BCNF	Complex dependency issues

8. Advantages of Normalization

Advantage	Explanation
Less redundancy	Avoid duplicate data
Better consistency	Data remains accurate
Easy maintenance	Updates become easier
Better storage	Less wasted space
Improved integrity	Relationships are clear

9. Disadvantages of Normalization

Disadvantage	Explanation
More tables	Database becomes complex
More joins required	Queries may become slower
Design complexity	Requires planning

10. Normalization vs Denormalization

Normalization	Denormalization
Splits tables	Combines tables
Reduces redundancy	Increases redundancy
Better consistency	Faster reading
Used in transactional systems	Used in analytics systems

Interview Questions

Q1. What is normalization?

Answer:

Normalization is the process of organizing data into multiple related tables to reduce redundancy and improve data integrity.

Q2. What are anomalies?

Types:

1. Insert anomaly
2. Update anomaly
3. Delete anomaly

Q3. Difference between 2NF and 3NF?

2NF	3NF
Removes partial dependency	Removes transitive dependency
Requires 1NF	Requires 2NF

Q4. What is functional dependency?

A relationship where one attribute determines another attribute.

Example:

`Student_ID → Student_Name`

Quick Revision Notes

Normalization:

Goal:

Remove redundancy + improve consistency

Anomalies:

1. Insert
2. Update
3. Delete

Normal Forms:

1NF:

Atomic values

2NF:

1NF + No partial dependency

3NF:

2NF + No transitive dependency

BCNF:

Every determinant is candidate key

Normalization = Split tables

Denormalization = Combine tables

Next Topic:

SQL (Structured Query Language)

We will cover:

- What is SQL?
- SQL commands
- SQL syntax
- Data types
- Creating databases/tables
- Insert, Update, Delete
- Select queries
- SQL Server basics
- Oracle basics
- Interview questions

Proceeding step-by-step.

Ok next

Module 4: Database Systems

Topic 7: SQL (Structured Query Language)

1. What is SQL?

SQL (Structured Query Language) is a standard programming language used to communicate with relational databases.

It is used to:

- Create databases
- Create tables
- Insert data
- Retrieve data
- Update data
- Delete data
- Control access
- Manage transactions

In simple words:

SQL is the language used to talk with a database.

2. Real-World Example

Suppose Amazon stores:

- Users
- Products
- Orders
- Payments

When you search for a product:

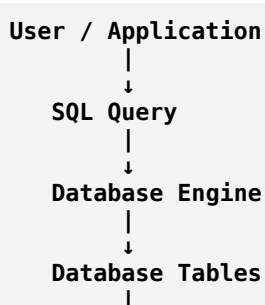
"Show all laptops under ₹50000"

The application sends an SQL query:

```
SQL
SELECT *
FROM Products
WHERE Category='Laptop'
AND Price < 50000;
```

Database returns matching products.

3. SQL Working Architecture



↓
Result

4. Features of SQL

1. Simple Language

SQL uses English-like commands.

Example:

```
SQL
SELECT Name
FROM Student;
```

2. Standard Language

SQL is supported by:

- Oracle
- MySQL
- SQL Server
- PostgreSQL

3. Supports Data Security

Controls:

- Users
- Permissions
- Access

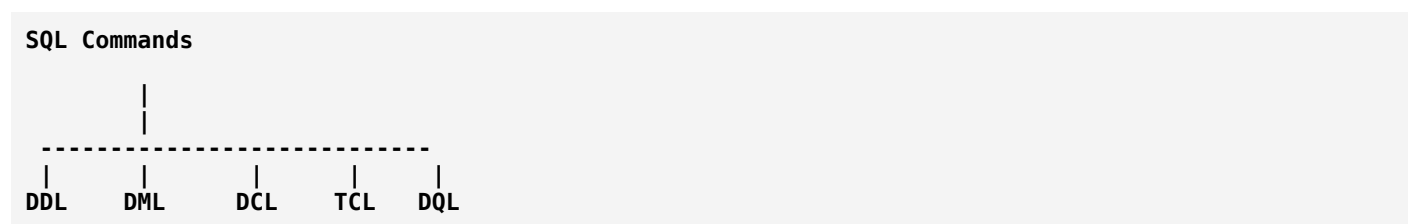
4. Handles Large Data

Used for millions of records in:

- Banking
- E-commerce
- Healthcare

5. Types of SQL Commands

SQL commands are divided into five categories:



1. DDL (Data Definition Language)

Used to define database structure.

Commands:

- CREATE
- ALTER
- DROP
- TRUNCATE

Example:

Create table:

```

SQL
CREATE TABLE Student(
  ID INT,
  Name VARCHAR(50),
  Age INT
);
  
```

2. DML (Data Manipulation Language)

Used to modify data inside tables.

Commands:

- INSERT
- UPDATE
- DELETE

Example:

Insert:

```

SQL
INSERT INTO Student
VALUES(101, 'Rahul', 20);
  
```

Update:

```

SQL
  
```

```
UPDATE Student
SET Age=21
WHERE ID=101;
```

Delete:

```
SQL
DELETE FROM Student
WHERE ID=101;
```

3. DQL (Data Query Language)

Used to retrieve data.

Main command:

- SELECT

Example:

```
SQL
SELECT *
FROM Student;
```

Output:

ID	Name	Age
101	Rahul	20

4. DCL (Data Control Language)

Used for permissions and access control.

Commands:

- GRANT
- REVOKE

Example:

Give permission:

```
SQL
GRANT SELECT
ON Student
TO User1;
```

Remove permission:

```
SQL
REVOKE SELECT
ON Student
```

```
FROM User1;
```

5. TCL (Transaction Control Language)

Used to manage transactions.

Commands:

- COMMIT
- ROLLBACK
- SAVEPOINT

Example:

Transaction:

```
SQL
UPDATE Account
SET Balance=9000
WHERE ID=101;

COMMIT;
```

6. SQL Data Types

Data types define what kind of data a column can store.

Numeric Types

INT

Stores integers.

Example:

```
SQL
Age INT;
```

FLOAT

Stores decimal values.

Example:

```
SQL
CGPA FLOAT;
```

Character Types

CHAR

Fixed length.

Example:

```
SQL
Gender CHAR(1);
```

Stores:

```
M
F
```

VARCHAR

Variable length.

Example:

```
SQL
Name VARCHAR(50);
```

Date Types

DATE

Stores dates.

Example:

```
SQL
DOB DATE;
```

7. Creating Database

Example:

```
SQL
CREATE DATABASE College;
```

Select database:

```
SQL
USE College;
```

8. Creating Table

Example:


```
SQL
CREATE TABLE Student(
Student_ID INT PRIMARY KEY,
Name VARCHAR(50),
Branch VARCHAR(20),
Age INT
);
```

Created table:

Student_ID	Name	Branch	Age

9. INSERT Command

Used to add records.

Syntax:

```
SQL
INSERT INTO table_name
VALUES(values);
```

Example:

```
SQL
INSERT INTO Student
VALUES
(101, 'Rahul', 'CSE', 20);
```

Multiple rows:

```
SQL
INSERT INTO Student
VALUES
(102, 'Priya', 'ECE', 21),
(103, 'Amit', 'IT', 22);
```

10. SELECT Command

Used to retrieve data.

Select All Columns

```
SQL
SELECT *
FROM Student;
```

Select Specific Columns

```
SQL
SELECT Name, Branch
FROM Student;
```

Output:

Name	Branch
Rahul	CSE

11. WHERE Clause

Used for filtering.

Example:

```
SQL
SELECT *
FROM Student
WHERE Branch='CSE';
```

12. Operators in SQL

Comparison Operators

Operator	Meaning
=	Equal
>	Greater
<	Less
>=	Greater or equal
<=	Less or equal

Example:

```
SQL
SELECT *
FROM Student
WHERE Age > 20;
```

Logical Operators

AND

Both conditions true.

```
SQL
SELECT *
FROM Student
WHERE Branch='CSE'
AND Age>20;
```

OR

Any condition true.

```
SQL
SELECT *
FROM Student
WHERE Branch='CSE'
OR Branch='ECE';
```

NOT

Reverse condition.

```
SQL
SELECT *
FROM Student
WHERE NOT Branch='CSE';
```

13. ORDER BY

Sorts data.

Ascending:

```
SQL
SELECT *
FROM Student
ORDER BY Age ASC;
```

Descending:

```
SQL
SELECT *
FROM Student
ORDER BY Age DESC;
```

14. DISTINCT

Removes duplicate values.

Example:

Table:

Branch
CSE
ECE

Branch

CSE

Query:

```
SQL
SELECT DISTINCT Branch
FROM Student;
```

Output:

```
CSE
ECE
```

15. SQL Example Database

Student Table:

ID	Name	Branch	Marks
1	Rahul	CSE	90
2	Priya	ECE	85
3	Amit	CSE	95

Query:

Find CSE students:

```
SQL
SELECT Name
FROM Student
WHERE Branch='CSE';
```

Result:

```
Rahul
Amit
```

SQL Interview Questions

Q1. What is SQL?

Answer:

SQL is a programming language used to create, manipulate, and retrieve data from relational databases.

Q2. Difference between DELETE, DROP, and TRUNCATE?

DELETE	TRUNCATE	DROP
Deletes rows	Deletes all rows	Deletes complete table
Can use WHERE	No WHERE	Removes structure
Can rollback	Limited rollback	Cannot rollback

Q3. Difference between SQL and MySQL?

SQL	MySQL
Language	Database software
Used to write queries	Executes SQL queries

Q4. What is a query?

A query is a request sent to the database to perform an operation.

Example:

```
SQL
SELECT * FROM Employee;
```

Quick Revision Notes

SQL:

Used for:

- Create database
- Insert data
- Retrieve data
- Update data
- Delete data

SQL Categories:

DDL:

CREATE
ALTER
DROP
TRUNCATE

DML:

INSERT
UPDATE
DELETE

DQL:

SELECT

DCL:

GRANT
REVOKE

TCL:
COMMIT
ROLLBACK
SAVEPOINT

Next Topic:

DDL (Data Definition Language)

We will cover:

- CREATE
- ALTER
- DROP
- TRUNCATE
- Constraints
- SQL Server examples
- Oracle examples
- Practical queries
- Interview questions

Proceeding step-by-step.

Ok next

Module 4: Database Systems

Topic 8: DDL (Data Definition Language)

1. What is DDL?

DDL (Data Definition Language) is a set of SQL commands used to **define and modify the structure of database objects**.

Database objects include:

- Database
- Tables
- Views
- Indexes
- Schemas

In simple words:

DDL is used to create and change the design/structure of a database, not the data inside it.

2. DDL Commands

Main DDL commands:

Command	Purpose
CREATE	Creates database objects
ALTER	Modifies existing objects
DROP	Deletes database objects
TRUNCATE	Removes all records from a table
RENAME	Changes object name

1. CREATE Command

Purpose:

Used to create:

- Database
- Table
- View
- Index

Creating Database

SQL Server:

```
SQL
CREATE DATABASE College;
```

Oracle:

```
SQL
CREATE DATABASE College;
```

Creating Table

Syntax:

```
SQL
CREATE TABLE table_name(
column1 datatype,
column2 datatype
);
```

Example:

```
SQL
CREATE TABLE Student(
Student_ID INT PRIMARY KEY,
Name VARCHAR(50),
Age INT,
Branch VARCHAR(20)
);
```

Table created:

Student_ID	Name	Age	Branch

2. ALTER Command

Purpose:

Used to modify an existing table.

Operations:

- Add column
- Modify column
- Drop column
- Rename column

A) Add Column

Syntax:

```
SQL
ALTER TABLE table_name
ADD column_name datatype;
```

Example:


```
SQL
ALTER TABLE Student
ADD Email VARCHAR(100);
```

Before:

ID	Name
101	Rahul

After:

ID	Name	Email
101	Rahul	

B) Modify Column

Used to change datatype or size.

Example:

SQL Server:

```
SQL
ALTER TABLE Student
ALTER COLUMN Name VARCHAR(100);
```

Oracle:

```
SQL
ALTER TABLE Student
MODIFY Name VARCHAR2(100);
```

C) Drop Column

Removes a column.

Example:

```
SQL
ALTER TABLE Student
DROP COLUMN Email;
```

D) Rename Column

SQL Server:

```
SQL
EXEC sp_rename
'Student.Name',
```

```
'Student_Name',  
'COLUMN';
```

Oracle:

```
SQL  
ALTER TABLE Student  
RENAME COLUMN Name TO Student_Name;
```

3. DROP Command

Purpose:

Deletes database objects permanently.

Example:

```
SQL  
DROP TABLE Student;
```

Result:

Table completely removed.

Before:

```
Student Table  
  |  
  ↓  
DROP TABLE  
  |  
  ↓  
No table exists
```

DROP Database

```
SQL  
DROP DATABASE College;
```

Deletes the entire database.

4. TRUNCATE Command

Purpose:

Removes all records from a table but keeps the table structure.

Example:

Before:

Student Table:

ID	Name
1	Rahul
2	Priya

Query:

```
SQL
TRUNCATE TABLE Student;
```

After:

ID	Name
Empty	

Structure remains:

```
Student Table
|
|
|
↓
No Data
```

5. RENAME Command

Used to change object names.

Example:

Oracle:

```
SQL
RENAME Student TO Students;
```

6. DDL Constraints

Constraints are rules applied to columns to maintain data accuracy.

Main constraints:

1. PRIMARY KEY
2. FOREIGN KEY
3. UNIQUE
4. NOT NULL
5. CHECK
6. DEFAULT

1. PRIMARY KEY Constraint

Ensures:

- Unique values
- No NULL values

Example:

```
SQL
CREATE TABLE Student(
ID INT PRIMARY KEY,
Name VARCHAR(50)
);
```

2. FOREIGN KEY Constraint

Creates relationship between tables.

Example:

Department:

```
SQL
CREATE TABLE Department(
Dept_ID INT PRIMARY KEY,
Dept_Name VARCHAR(50)
);
```

Student:

```
SQL
CREATE TABLE Student(
ID INT PRIMARY KEY,
Name VARCHAR(50),
Dept_ID INT,
FOREIGN KEY(Dept_ID)
REFERENCES Department(Dept_ID)
);
```

3. UNIQUE Constraint

No duplicate values allowed.

Example:

```
SQL
```

```
CREATE TABLE Employee(
  ID INT,
  Email VARCHAR(100) UNIQUE
);
```

Invalid:

Email
a@gmail.com
a@gmail.com

4. NOT NULL Constraint

Column cannot have empty values.

Example:

```
SQL
CREATE TABLE Student(
  ID INT,
  Name VARCHAR(50) NOT NULL
);
```

Invalid:

Name = NULL

5. CHECK Constraint

Ensures values satisfy a condition.

Example:

Age should be greater than 18:

```
SQL
CREATE TABLE Student(
  ID INT,
  Age INT CHECK(Age >= 18)
);
```

Invalid:

Age = 15

6. DEFAULT Constraint

Assigns default value.

Example:

```
SQL
CREATE TABLE Employee(
  ID INT,
  Country VARCHAR(50)
  DEFAULT 'India'
);
```

Insert:

```
SQL
INSERT INTO Employee(ID)
VALUES(101);
```

Result:

ID	Country
101	India

DDL vs DML

DDL	DML
Defines structure	Manipulates data
CREATE, ALTER, DROP	INSERT, UPDATE, DELETE
Works on database objects	Works on records
Usually auto committed	Can be rolled back

DELETE vs TRUNCATE vs DROP

Feature	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
Deletes	Selected rows	All rows	Entire table
WHERE clause	Allowed	Not allowed	Not allowed
Structure remains	Yes	Yes	No
Rollback	Possible	Usually not	Not possible
Speed	Slow	Fast	Fast

Practical Example

Create Employee Management Database:

```
SQL
CREATE DATABASE Company;
```

Create table:

```
SQL
CREATE TABLE Employee(
Emp_ID INT PRIMARY KEY,
Name VARCHAR(50) NOT NULL,
Salary INT CHECK(Salary>0),
Department VARCHAR(30)
);
```

Add column:

```
SQL
ALTER TABLE Employee
ADD Email VARCHAR(100);
```

Remove table data:

```
SQL
TRUNCATE TABLE Employee;
```

Delete table:

```
SQL
DROP TABLE Employee;
```

Interview Questions

Q1. What is DDL?

Answer:

DDL is a set of SQL commands used to create, modify, and delete database structures.

Q2. Is TRUNCATE a DDL command?

Yes. TRUNCATE is classified as DDL because it changes the storage structure by removing all records.

Q3. Difference between DROP and TRUNCATE?

DROP:

- Removes table completely.

TRUNCATE:

- Removes only data.
- Keeps table structure.

Q4. Can we rollback DDL commands?

Generally no, because most databases automatically commit DDL operations.

Quick Revision Notes

DDL:

CREATE

→ Create database/table

ALTER

→ Modify structure

DROP

→ Delete object permanently

TRUNCATE

→ Remove all records

RENAME

→ Change object name

Constraints:

PRIMARY KEY

FOREIGN KEY

UNIQUE

NOT NULL

CHECK

DEFAULT

Next Topic:

DML (Data Manipulation Language)

We will cover:

- INSERT
- UPDATE
- DELETE
- MERGE
- Data manipulation examples

- SQL Server practice queries
- Oracle syntax
- Interview questions

Proceeding step-by-step.

Ok next

Module 4: Database Systems

Topic 9: DML (Data Manipulation Language)

1. What is DML?

DML (Data Manipulation Language) is a set of SQL commands used to **insert, modify, and delete data inside database tables**.

In simple words:

DML is used to work with the actual data stored in tables.

DDL changes the **structure**.

DML changes the **data**.

2. DML Commands

Command	Purpose
INSERT	Add new records
UPDATE	Modify existing records
DELETE	Remove records
MERGE	Insert/update data conditionally

1. INSERT Command

Purpose:

Used to add new rows into a table.

Syntax:

```
SQL
INSERT INTO table_name
VALUES(value1, value2, ...);
```

Example

Create table:

```
SQL
CREATE TABLE Student(
ID INT,
Name VARCHAR(50),
Branch VARCHAR(20),
Age INT
);
```

Insert data:

```
SQL
INSERT INTO Student
VALUES(101, 'Rahul', 'CSE', 20);
```

Table:

ID	Name	Branch	Age
101	Rahul	CSE	20

Insert Multiple Records

```
SQL
INSERT INTO Student
VALUES
(102, 'Priya', 'ECE', 21),
(103, 'Amit', 'IT', 22),
(104, 'John', 'CSE', 20);
```

Result:

ID	Name	Branch	Age
101	Rahul	CSE	20
102	Priya	ECE	21
103	Amit	IT	22
104	John	CSE	20

Insert Specific Columns

Suppose table:

Student
ID
Name
Age
Branch

Query:

```
SQL
INSERT INTO Student(ID,Name)
VALUES(105,'Kiran');
```

Result:

ID	Name	Age	Branch
105	Kiran	NULL	NULL

2. UPDATE Command

Purpose:

Used to modify existing records.

Syntax:

```
SQL
UPDATE table_name
SET column=value
WHERE condition;
```

Example

Before:

ID	Name	Age
101	Rahul	20

Query:

```
SQL
UPDATE Student
SET Age=21
WHERE ID=101;
```

After:

ID	Name	Age
101	Rahul	21

Updating Multiple Columns

```
SQL
UPDATE Student
SET
Age=22,
Branch='IT'
WHERE ID=101;
```

Important: WHERE Clause

Without WHERE:

```
SQL
UPDATE Student
SET Age=25;
```

All students' age becomes 25.

Example:

Before:

Name	Age
Rahul	20
Priya	21

After:

Name	Age
Rahul	25
Priya	25

So always use WHERE carefully.

3. DELETE Command

Purpose:

Removes records from a table.

Syntax:

```
SQL
DELETE FROM table_name
WHERE condition;
```

Example:

Before:

ID	Name
101	Rahul
102	Priya

Query:

```
SQL
DELETE FROM Student
WHERE ID=101;
```

After:

ID	Name
102	Priya

Delete All Records

```
SQL
DELETE FROM Student;
```

Result:

Table becomes empty.

But structure remains.

4. MERGE Command

Purpose:

MERGE performs:

- INSERT if record does not exist
- UPDATE if record exists

It is also called **UPSERT**.

Example:

Existing Employee table:

ID	Name	Salary
101	Rahul	50000

New data:

ID	Name	Salary
101	Rahul	60000
102	Priya	55000

MERGE operation:

For ID 101:

Already exists
→ UPDATE salary

For ID 102:

New record
→ INSERT

SQL Server MERGE Example

```
SQL
MERGE Employee AS E
USING NewEmployee AS N
ON E.ID=N.ID
WHEN MATCHED THEN
UPDATE SET
E.Salary=N.Salary
WHEN NOT MATCHED THEN
INSERT VALUES
(N.ID,N.Name,N.Salary);
```

5. DML Transaction Behavior

DML operations can be controlled using TCL commands.

Example:

```
SQL
UPDATE Account
SET Balance=9000
WHERE ID=101;
```

Before saving permanently:

```
SQL
ROLLBACK;
```

Changes are cancelled.

Save permanently:

```
SQL
COMMIT;
```

6. DML with WHERE Clause

WHERE filters records.

Example:

Student table:

ID	Name	Branch
1	Rahul	CSE
2	Priya	ECE
3	Amit	CSE

Update only CSE students:

```
SQL
UPDATE Student
SET Branch='Computer Science'
WHERE Branch='CSE';
```

7. DML Operators

Comparison Operators

Example:

Find students older than 20:

```
SQL
SELECT *
FROM Student
WHERE Age > 20;
```

BETWEEN

```
SQL
SELECT *
FROM Student
```

WHERE Age BETWEEN 18 AND 22;

IN

```
SQL
SELECT *
FROM Student
WHERE Branch IN('CSE', 'ECE');
```

LIKE

Pattern matching.

Find names starting with R:

```
SQL
SELECT *
FROM Student
WHERE Name LIKE 'R%';
```

Examples:

Pattern	Meaning
R%	Starts with R
%R	Ends with R
%R%	Contains R

8. INSERT vs UPDATE vs DELETE

INSERT	UPDATE	DELETE
Adds new records	Changes existing records	Removes records
Creates rows	Modifies rows	Deletes rows
No WHERE required	Usually uses WHERE	Uses WHERE

9. DML vs DDL

DML	DDL
Works on data	Works on structure
INSERT, UPDATE, DELETE	CREATE, ALTER, DROP
Can rollback	Usually auto committed
Changes table contents	Changes table design

Practical Example: Employee Management

Insert Employee

```
SQL
INSERT INTO Employee
VALUES
(1, 'Rahul', 50000);
```

Increase Salary

```
SQL
UPDATE Employee
SET Salary=60000
WHERE ID=1;
```

Remove Employee

```
SQL
DELETE FROM Employee
WHERE ID=1;
```

Interview Questions

Q1. What is DML?

Answer:

DML is a group of SQL commands used to insert, update, and delete data stored in database tables.

Q2. Difference between DELETE and TRUNCATE?

DELETE	TRUNCATE
DML command	DDL command
Deletes selected rows	Deletes all rows
WHERE allowed	WHERE not allowed
Can rollback	Usually cannot rollback

Q3. What happens if UPDATE is executed without WHERE?

All rows in the table will be updated.

Q4. What is MERGE?

MERGE combines INSERT and UPDATE operations based on a condition.

Quick Revision Notes

DML:

INSERT

→ Add records

UPDATE

→ Modify records

DELETE

→ Remove records

MERGE

→ Insert + Update

Important:

Always use WHERE with UPDATE and DELETE.

Transaction control:

COMMIT

ROLLBACK

Next Topic:

DCL (Data Control Language)

We will cover:

- GRANT
- REVOKE
- User permissions
- Database security
- Roles
- SQL Server and Oracle examples
- Interview questions

Proceeding step-by-step.

ok next

Module 4: Database Systems

Topic 10: DCL (Data Control Language)

1. What is DCL?

DCL (Data Control Language) is a set of SQL commands used to **control access and permissions on database objects**.

It manages:

- Who can access data
- What operations a user can perform
- Database security

In simple words:

DCL decides who can do what inside a database.

2. Why Do We Need DCL?

In real-world applications, every user should not have full access.

Example: Banking System

Database Administrator (DBA)

Can:

- Create tables
 - Delete databases
 - Manage users
-

Bank Employee

Can:

- View customer details
- Update account information

Cannot:

- Delete database
-

Customer

Can:

- View own account

Cannot:

- Access other customers' data

DCL provides this security.

3. DCL Commands

The main DCL commands are:

Command	Purpose
GRANT	Gives permissions
REVOKE	Removes permissions

1. GRANT Command

Definition:

GRANT gives specific privileges to users.

Syntax:

```
SQL
GRANT privilege
ON object
TO user;
```

Example 1: Give SELECT Permission

Suppose we have:

Student Table

Give permission:

```
SQL
GRANT SELECT
ON Student
TO User1;
```

Now User1 can view Student data.

Example 2: Give Multiple Permissions

```
SQL
GRANT SELECT, INSERT, UPDATE
ON Employee
```

TO Manager;

Manager can:

- View employees
- Add employees
- Update employees

2. REVOKE Command

Definition:

REVOKE removes previously granted permissions.

Syntax:

```
SQL
REVOKE privilege
ON object
FROM user;
```

Example:

Remove SELECT permission:

```
SQL
REVOKE SELECT
ON Student
FROM User1;
```

Now User1 cannot view Student table.

4. Types of Privileges

Privileges are mainly divided into two types:

1. System Privileges
2. Object Privileges

1. System Privileges

Permissions to perform database-level operations.

Examples:

- Create database

- Create user
- Create table

Example:

```
SQL
GRANT CREATE TABLE
TO User1;
```

2. Object Privileges

Permissions on specific database objects.

Objects:

- Tables
- Views
- Procedures

Examples:

```
SQL
GRANT SELECT
ON Employee
TO User1;
```

5. Common SQL Privileges

Privilege	Purpose
SELECT	Read data
INSERT	Add records
UPDATE	Modify records
DELETE	Remove records
EXECUTE	Run procedures
REFERENCES	Create foreign keys

6. Database Users and Roles

Managing individual permissions becomes difficult.

Example:

A company has:

- 500 employees
- 50 managers

- 10 administrators

Instead of giving permissions one by one, we use **roles**.

Role

A role is a collection of permissions.

Example:

Manager Role

```
|
|--- SELECT
|--- INSERT
|--- UPDATE
```

Assign role:

```
SQL
GRANT Manager_Role
TO Employee1;
```

7. GRANT WITH OPTION

Allows a user to give permissions to other users.

Example:

```
SQL
GRANT SELECT
ON Student
TO User1
WITH GRANT OPTION;
```

Now User1 can:

- View Student table
- Give SELECT permission to others

8. REVOKE with Cascade

When a user loses permission, permissions given by that user may also be removed.

Example:

```
Admin
|
| Gives SELECT
|
User1
|
Gives SELECT
```

User2

If Admin revokes User1:

```
SQL
REVOKE SELECT
FROM User1;
```

User2 may also lose access.

9. DCL Example: Company Database

Tables:

```
Employee
Department
Salary
```

Create User

Oracle:

```
SQL
CREATE USER Rahul
IDENTIFIED BY password;
```

Give Access

```
SQL
GRANT SELECT
ON Employee
TO Rahul;
```

Allow Updates

```
SQL
GRANT UPDATE
ON Employee
TO Rahul;
```

Remove Access

```
SQL
REVOKE UPDATE
ON Employee
FROM Rahul;
```


10. DCL in SQL Server

Create login:

```
SQL
CREATE LOGIN user1
WITH PASSWORD='Password@123';
```

Create user:

```
SQL
CREATE USER user1
FOR LOGIN user1;
```

Grant permission:

```
SQL
GRANT SELECT
ON Employee
TO user1;
```

11. DCL in Oracle

Create user:

```
SQL
CREATE USER user1
IDENTIFIED BY password;
```

Grant permission:

```
SQL
GRANT SELECT
ON Employee
TO user1;
```

Remove permission:

```
SQL
REVOKE SELECT
ON Employee
FROM user1;
```

DCL vs DDL vs DML

Feature	DDL	DML	DCL
Purpose	Structure	Data	Security
Commands	CREATE, ALTER	INSERT, UPDATE	GRANT, REVOKE
Works on	Database objects	Records	Permissions
Users	Developers/DBA	Developers	DBA

Real-World Example

Hospital Database

Tables:

- Patients
- Doctors
- Medicines

Permissions:

Doctor:

```
SQL
GRANT SELECT, UPDATE
ON Patients
TO Doctor;
```

Receptionist:

```
SQL
GRANT SELECT, INSERT
ON Patients
TO Receptionist;
```

Admin:

```
SQL
GRANT ALL
ON Patients
TO Admin;
```

Interview Questions

Q1. What is DCL?

Answer:

DCL is used to control user access and permissions in a database using commands like GRANT and REVOKE.

Q2. Difference between GRANT and REVOKE?

GRANT	REVOKE
Provides permissions	Removes permissions
Gives access	Restricts access

Q3. Who manages DCL operations?

Usually:

- Database Administrator (DBA)
-

Q4. What is a role?

A role is a collection of database privileges assigned to users.

Q5. What is the difference between system and object privileges?

System privilege:

- Database-level permission.

Object privilege:

- Permission on specific objects like tables or views.
-

Quick Revision Notes

DCL:
Controls database security

Commands:

GRANT
→ Give permissions

REVOKE
→ Remove permissions

Privileges:

SELECT
INSERT
UPDATE
DELETE
EXECUTE

Users:
DBA
Developers
End Users

Role:
Collection of permissions

Next Topic:

TCL (Transaction Control Language)

We will cover:

- Transactions
- COMMIT
- ROLLBACK
- SAVEPOINT
- ACID properties
- Transaction states
- Real-world banking examples
- Interview questions

Proceeding step-by-step.

Ok next

Module 4: Database Systems

Topic 11: TCL (Transaction Control Language)

1. What is TCL?

TCL (Transaction Control Language) is a set of SQL commands used to **manage transactions in a database**.

It controls:

- Saving changes permanently
- Undoing changes
- Creating checkpoints

In simple words:

TCL controls the execution of a group of SQL operations as a single unit.

2. What is a Transaction?

A **transaction** is a sequence of one or more SQL operations that perform a single logical task.

Example:

Bank Money Transfer

Transfer ₹5000 from Account A to Account B.

Steps:

Step 1:

Remove money from A:

```
SQL
UPDATE Account
SET Balance = Balance - 5000
WHERE Account_ID = 101;
```

Step 2:

Add money to B:

```
SQL
UPDATE Account
SET Balance = Balance + 5000
WHERE Account_ID = 102;
```

Both operations together form one transaction.

If one fails, the entire transaction should fail.

3. Transaction Properties (ACID)

A transaction must satisfy ACID properties.

ACID stands for:

- Atomicity
- Consistency
- Isolation
- Durability

1. Atomicity

Meaning:

A transaction is treated as a single unit.

Either:

- All operations succeed
- Or all operations fail

Example:

Money transfer:

```
Debit ₹5000
+
Credit ₹5000
```

If credit fails:

Debit should also be cancelled.

2. Consistency

Meaning:

Database must remain valid before and after a transaction.

Example:

Before transfer:

```
Total bank money = ₹1,00,000
```

After transfer:

```
Total bank money = ₹1,00,000
```

Money should not disappear.

3. Isolation

Meaning:

Multiple transactions should not interfere with each other.

Example:

Two users accessing the same account:

Transaction A:

```
Withdraw ₹5000
```

Transaction B:

```
Check Balance
```

Database handles them safely.

4. Durability

Meaning:

Once changes are committed, they are permanently stored.

Example:

After:

```
SQL
COMMIT;
```

Even if the server crashes:

Data remains.

4. TCL Commands

Main TCL commands:

Command	Purpose
COMMIT	Save changes permanently
ROLLBACK	Undo changes
SAVEPOINT	Create checkpoint

1. COMMIT Command

Purpose:

Permanently saves transaction changes.

Syntax:

```
SQL
COMMIT;
```

Example:

Initial table:

ID	Balance
101	10000

Transaction:

```
SQL
UPDATE Account
SET Balance=15000
WHERE ID=101;
```

Save:

```
SQL
COMMIT;
```

Now:

```
Balance = 15000
```

permanently saved.

2. ROLLBACK Command

Purpose:

Undo changes made in current transaction.

Syntax:

```
SQL
ROLLBACK;
```

Example:

Before:

ID	Balance
101	10000

Update:

```
SQL
UPDATE Account
SET Balance=5000
WHERE ID=101;
```

Mistake detected.

Undo:

```
SQL
ROLLBACK;
```

Result:

ID	Balance
101	10000

3. SAVEPOINT Command

Purpose:

Creates a temporary checkpoint inside a transaction.

Syntax:

```
SQL
SAVEPOINT savepoint_name;
```

Example:

```
SQL
UPDATE Account
SET Balance=9000
WHERE ID=101;

SAVEPOINT A;

UPDATE Account
SET Balance=8000
WHERE ID=101;

SAVEPOINT B;
```

Transaction state:

```
Start
|
Update 9000
|
SAVEPOINT A
|
Update 8000
|
SAVEPOINT B
```

Rollback to SAVEPOINT A:

```
SQL
ROLLBACK TO A;
```

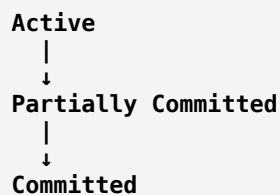
Result:

Balance returns to:

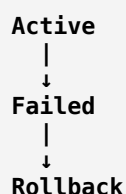
```
9000
```

5. Transaction States

A transaction goes through different states.



or



1. Active State

Transaction is executing.

Example:

```

SQL
UPDATE Account;
  
```

2. Partially Committed

Last operation executed.

Waiting for permanent save.

3. Committed

Changes permanently stored.

After:

```

SQL
COMMIT;
  
```

4. Failed State

Transaction cannot continue.

Example:

- Server crash

- Constraint violation

5. Rolled Back State

Database returns to previous state.

6. Auto Commit

Many databases automatically commit transactions.

Example:

SQL Server:

```
SQL
INSERT INTO Student
VALUES(101, 'Rahul');
```

May be automatically saved.

To control manually:

```
SQL
BEGIN TRANSACTION;
```

7. TCL Example

Employee Table:

ID	Salary
1	50000

Transaction:

```
SQL
BEGIN TRANSACTION;

UPDATE Employee
SET Salary=60000
WHERE ID=1;

SAVEPOINT Salary_Update;

UPDATE Employee
SET Salary=70000
WHERE ID=1;

ROLLBACK TO Salary_Update;

COMMIT;
```

Final Salary:

60000

8. COMMIT vs ROLLBACK

COMMIT	ROLLBACK
Saves changes	Cancels changes
Permanent	Temporary undo
Cannot reverse	Returns previous state
Used after successful transaction	Used after errors

9. DELETE vs ROLLBACK Example

Before:

ID	Name
1	Rahul
2	Priya

Delete:

```
SQL
DELETE FROM Student
WHERE ID=1;
```

Undo:

```
SQL
ROLLBACK;
```

Data comes back.

10. TCL in Real Applications

Banking

Transactions:

- Deposit
- Withdrawal
- Transfer

E-Commerce

Order placement:

```
Create Order
↓
Reduce Inventory
↓
Process Payment
```

If payment fails:

Rollback order creation.

Airline Reservation

Booking:

```
Reserve Seat
↓
Payment
↓
Confirm Ticket
```

Failure:

Rollback reservation.

Interview Questions

Q1. What is TCL?

Answer:

TCL is a group of SQL commands used to manage database transactions.

Q2. What are ACID properties?

Answer:

- Atomicity
 - Consistency
 - Isolation
 - Durability
-

Q3. Difference between COMMIT and ROLLBACK?

COMMIT permanently saves changes.

ROLLBACK cancels uncommitted changes.

Q4. What is SAVEPOINT?

SAVEPOINT creates a checkpoint inside a transaction to allow partial rollback.

Q5. Why are transactions important?

They maintain:

- Data accuracy
 - Reliability
 - Consistency
-

Quick Revision Notes

TCL:

Transaction:
Group of SQL operations

ACID:

A → Atomicity
C → Consistency
I → Isolation
D → Durability

Commands:

COMMIT
→ Save permanently

ROLLBACK
→ Undo changes

SAVEPOINT
→ Create checkpoint

Next Topic:

SQL Joins

We will cover:

- What are joins?
- Why joins are needed
- INNER JOIN
- LEFT JOIN
- RIGHT JOIN

- FULL JOIN
- CROSS JOIN
- SELF JOIN
- Real-world examples
- SQL queries
- Interview questions

Proceeding step-by-step.

Ok next

Module 4: Database Systems

Topic 12: SQL Joins

1. What are SQL Joins?

A **JOIN** is an SQL operation used to **combine data from two or more tables** based on a related column.

In simple words:

Joins help us retrieve related information stored in different tables.

2. Why Do We Need Joins?

In a properly designed database, data is divided into multiple tables using normalization.

Example:

Student Table

Student_ID	Name	Dept_ID
101	Rahul	10
102	Priya	20
103	Amit	10

Department Table

Dept_ID	Dept_Name
10	CSE

Dept_ID	Dept_Name
20	ECE

If we want:

"Show student names with department names"

We need to combine both tables.

Result:

Name	Department
Rahul	CSE
Priya	ECE
Amit	CSE

This is done using JOIN.

3. Types of SQL Joins

Main types:

1. INNER JOIN
2. LEFT JOIN
3. RIGHT JOIN
4. FULL OUTER JOIN
5. CROSS JOIN
6. SELF JOIN

1. INNER JOIN

Definition:

Returns only the records that have matching values in both tables.

Syntax:

```
SQL
SELECT columns
FROM Table1
INNER JOIN Table2
ON Table1.column = Table2.column;
```

Example:

Student Table:

ID	Name	Dept_ID
101	Rahul	10
102	Priya	20
103	Amit	30

Department Table:

Dept_ID	Dept_Name
10	CSE
20	ECE

Query:

```
SQL
SELECT
Student.Name,
Department.Dept_Name
FROM Student
INNER JOIN Department
ON Student.Dept_ID = Department.Dept_ID;
```

Output:

Name	Dept_Name
Rahul	CSE
Priya	ECE

Notice:

Amit is removed because Dept_ID 30 does not exist.

2. LEFT JOIN (LEFT OUTER JOIN)

Definition:

Returns:

- All records from left table
- Matching records from right table

If no match exists:

NULL values are returned.

Syntax:

```
SQL
SELECT columns
FROM Table1
LEFT JOIN Table2
ON condition;
```

Example:

Student:

ID	Name	Dept_ID
101	Rahul	10
102	Priya	20
103	Amit	30

Department:

Dept_ID	Dept_Name
10	CSE
20	ECE

Query:

```
SQL
SELECT
Student.Name,
Department.Dept_Name
FROM Student
LEFT JOIN Department
ON Student.Dept_ID = Department.Dept_ID;
```

Output:

Name	Dept_Name
Rahul	CSE
Priya	ECE
Amit	NULL

3. RIGHT JOIN (RIGHT OUTER JOIN)

Definition:

Returns:

- All records from right table
- Matching records from left table

Example:

Student:

ID	Name	Dept_ID
101	Rahul	10
102	Priya	20

Department:

Dept_ID	Dept_Name
10	CSE
20	ECE
30	MECH

Query:

```
SQL
SELECT
Student.Name,
Department.Dept_Name
FROM Student
RIGHT JOIN Department
ON Student.Dept_ID = Department.Dept_ID;
```

Output:

Name	Dept_Name
Rahul	CSE
Priya	ECE
NULL	MECH

4. FULL OUTER JOIN

Definition:

Returns:

- All records from left table
- All records from right table

Matched records are combined.

Unmatched records contain NULL.

Example:

Output:

Name	Department
Rahul	CSE
Priya	ECE
Amit	NULL
NULL	MECH

Syntax:

```
SQL
SELECT *
FROM Student
FULL OUTER JOIN Department
ON Student.Dept_ID = Department.Dept_ID;
```

5. CROSS JOIN

Definition:

Returns the Cartesian product of two tables.

Every row of first table combines with every row of second table.

Example:

Student:

Name
Rahul
Priya

Course:

Course
Java
Python

Result:

Name	Course
Rahul	Java
Rahul	Python
Priya	Java
Priya	Python

Number of rows:

Student rows × Course rows
2 × 2 = 4

Syntax:

```
SQL
SELECT *
FROM Student
CROSS JOIN Course;
```

6. SELF JOIN

Definition:

A table joined with itself is called SELF JOIN.

Used when records in the same table have relationships.

Example:

Employee Table:

Emp_ID	Name	Manager_ID
1	John	NULL
2	Alex	1
3	Bob	1

Find employee and manager names:

```
SQL
SELECT
E.Name AS Employee,
M.Name AS Manager
FROM Employee E
JOIN Employee M
ON E.Manager_ID = M.Emp_ID;
```

Output:

Employee	Manager
Alex	John
Bob	John

7. Join Comparison Table

Join Type	Returns
INNER JOIN	Only matching rows
LEFT JOIN	All left + matching right
RIGHT JOIN	All right + matching left
FULL JOIN	Everything from both tables
CROSS JOIN	All combinations
SELF JOIN	Table joined with itself

8. JOIN vs UNION

JOIN	UNION
Combines columns	Combines rows
Uses related columns	Requires same number of columns
Works horizontally	Works vertically

Example:

JOIN:

```
Student + Department
Name + Dept_Name
```

UNION:

```
Table A rows
+
Table B rows
```

9. Real-World JOIN Examples

E-commerce

Tables:

Customer:

```
Customer_ID
Name
```

Orders:

```
Order_ID
Customer_ID
Amount
```

Query:

"Show customer orders"

```
SQL
SELECT
Customer.Name,
Orders.Amount

FROM Customer
JOIN Orders

ON Customer.Customer_ID =
Orders.Customer_ID;
```

Banking

Customer table:

```
Customer_ID
Name
```

Account table:

```
Account_ID
Customer_ID
Balance
```

JOIN gives customer account details.

10. Common Join Mistakes

1. Missing ON Condition

Wrong:

```
SQL
SELECT *
FROM Student
JOIN Department;
```

Creates unwanted Cartesian product.

2. Wrong Join Column

Wrong:

```
SQL
ON Student.ID = Department.ID
```

Correct:

```
SQL
ON Student.Dept_ID = Department.Dept_ID
```

Interview Questions

Q1. What is a JOIN?

Answer:

JOIN is an SQL operation used to combine data from multiple tables using a related column.

Q2. Difference between INNER JOIN and LEFT JOIN?

INNER JOIN	LEFT JOIN
Only matching records	All left records + matches
Unmatched rows removed	Unmatched rows shown as NULL

Q3. Which join returns all records from both tables?

Answer:

FULL OUTER JOIN

Q4. What is SELF JOIN?

A join where a table is joined with itself.

Example:

Employee → Manager relationship.

Quick Revision Notes

SQL JOIN:

Purpose:
Combine tables

Types:

INNER JOIN
→ Matching data only

LEFT JOIN
→ All left + matching right

RIGHT JOIN
→ All right + matching left

FULL JOIN
→ Everything

CROSS JOIN
→ All combinations

SELF JOIN
→ Table with itself

Next Topic:

SQL Views

We will cover:

- What is a View?
- Creating Views
- Types of Views
- Advantages
- Views vs Tables
- Updatable Views
- Real-world examples
- Interview questions

Proceeding step-by-step.

Ok next

Module 4: Database Systems

Topic 13: SQL Views

1. What is a View?

A **View** is a virtual table created from the result of an SQL query.

A view does **not store data permanently** (normally). It stores the SQL query definition and displays data from the underlying tables.

In simple words:

A View is a saved SQL query that behaves like a table.

2. Real-World Example

Consider a company database.

Employee Table:

Emp_ID	Name	Department	Salary
101	Rahul	CSE	60000
102	Priya	ECE	50000
103	Amit	IT	70000

The HR department can see:

- Name
- Department

But employees should not see salaries.

Instead of creating a new table, we create a View.

Employee View

Name	Department
Rahul	CSE
Priya	ECE
Amit	IT

Salary remains hidden.

3. Why Do We Use Views?

1. Security

Views hide sensitive information.

Example:

Bank database:

Original table:

Account_No	Name	Balance	PIN
101	Rahul	50000	1234

Customer view:

Account_No	Name	Balance
101	Rahul	50000

PIN is hidden.

2. Simplify Complex Queries

Suppose we frequently use:

```
SQL
SELECT
Student.Name,
Department.Dept_Name
FROM Student
JOIN Department
ON Student.Dept_ID=Department.Dept_ID;
```

Instead of writing this repeatedly, create a view.

3. Data Abstraction

Users do not need to know:

- Table structure
- Complex joins
- Database design

They only access the view.

4. Maintain Consistency

Everyone gets the same predefined data.

4. Creating a View

Syntax:

```
SQL
CREATE VIEW view_name AS

SELECT columns

FROM table_name

WHERE condition;
```

Example:

Employee Table:

```
Employee
-----
ID
Name
Department
Salary
```

Create view:

```
SQL
CREATE VIEW Employee_View AS

SELECT
Name,
Department

FROM Employee;
```

Now:

```
SQL
SELECT *
FROM Employee_View;
```

Output:

Name	Department
Rahul	CSE
Priya	ECE

5. View with WHERE Condition

Example:

Create view for CSE employees:

```
SQL
CREATE VIEW CSE_Employees AS

SELECT *
```

```
FROM Employee
WHERE Department='CSE';
```

Query:

```
SQL
SELECT *
FROM CSE_Employees;
```

Output:

Name	Department
Rahul	CSE

6. Updating Data Through Views

Some views are **updatable**.

Example:

View:

```
SQL
CREATE VIEW Student_View AS
SELECT ID,Name
FROM Student;
```

Update:

```
SQL
UPDATE Student_View
SET Name='Kiran'
WHERE ID=101;
```

Underlying Student table is updated.

7. Conditions for Updatable Views

A view is usually updatable if:

- It uses only one table
- No aggregate functions
- No GROUP BY
- No DISTINCT
- No JOIN

8. Types of Views

1. Simple View

Created from one table.

Example:

```
SQL
CREATE VIEW Student_View AS
SELECT Name, Age
FROM Student;
```

2. Complex View

Created using:

- Multiple tables
- Joins
- Functions

Example:

```
SQL
CREATE VIEW Student_Department AS
SELECT
Student.Name,
Department.Dept_Name
FROM Student
JOIN Department
ON Student.Dept_ID=Department.Dept_ID;
```

3. Materialized View

A materialized view stores the actual result physically.

Unlike normal views:

Normal View:

```
View
|
Query
|
Table
```

Materialized View:

```
View
|
Stored Result
```

Used in:

- Data warehouses
- Analytics systems

9. View Commands

Display View Data

```
SQL
SELECT *
FROM View_Name;
```

Modify View

```
SQL
CREATE OR ALTER VIEW Employee_View AS
SELECT Name, Department
FROM Employee;
```

(SQL Server)

Delete View

```
SQL
DROP VIEW Employee_View;
```

10. View vs Table

View	Table
Virtual table	Physical table
Stores query	Stores actual data
Uses less storage	Uses storage
Data comes from tables	Contains data directly
Improves security	Stores information

11. View vs Materialized View

View	Materialized View
Does not store data	Stores data physically
Always gets latest data	Needs refresh
Slower for complex queries	Faster retrieval
Less storage	More storage

12. Real-World Applications

Banking

Views:

- Customer account summary
- Transaction history

Hospital

Doctor View:

Shows:

- Patient name
- Medical history

Hides:

- Billing information

E-Commerce

Customer View:

Shows:

- Orders
- Products

Hides:

- Internal pricing information

13. Advantages of Views

Advantage	Explanation
Security	Hide sensitive data
Simplicity	Reduce complex queries
Data abstraction	Hide database complexity
Reusability	Use same query repeatedly
Consistency	Same output format

14. Disadvantages of Views

Disadvantage	Explanation
Performance issues	Complex views may be slow
Limited updates	Not all views are editable
Dependency	Changes in tables affect views

Interview Questions

Q1. What is a View?

Answer:

A View is a virtual table created using an SQL query that displays data from one or more tables.

Q2. Does a View store data?

Normal views do not store data. They store only the query definition.

Q3. Difference between View and Table?

A table stores actual data physically, while a view stores a query that retrieves data.

Q4. Why are views used?

- Security
- Simplifying queries
- Data abstraction
- Reusability

Q5. Can we update a View?

Yes, if it satisfies certain conditions like being based on a single table without complex operations.

Quick Revision Notes

VIEW:

Virtual table created from SQL query

Syntax:

```
CREATE VIEW name AS  
SELECT ...
```

Types:

1. Simple View
2. Complex View
3. Materialized View

Advantages:

Security
Abstraction
Simplification
Reusability

Commands:

```
CREATE VIEW  
SELECT FROM VIEW  
DROP VIEW
```

Next Topic:

Stored Procedures

We will cover:

- What are Stored Procedures?
- Procedure syntax
- Parameters
- Input/Output parameters
- Stored Procedure vs Function
- Advantages
- SQL Server & Oracle examples
- Interview questions

Proceeding step-by-step.

Ok next

Module 4: Database Systems

Topic 14: Stored Procedures

1. What is a Stored Procedure?

A **Stored Procedure** is a collection of SQL statements that are **stored permanently inside the database** and executed whenever required.

In simple words:

A Stored Procedure is a reusable SQL program stored in the database.

2. Why Do We Use Stored Procedures?

Suppose an application frequently needs to:

1. Check user login
2. Fetch customer details
3. Calculate salary
4. Generate reports

Instead of writing the same SQL queries repeatedly, we store them as procedures.

3. Normal Query vs Stored Procedure

Normal SQL Query

Application sends:

```
SQL
SELECT *
FROM Employee
WHERE ID=101;
```

Every time it runs, the database processes the query.

Stored Procedure

Create once:

```
SQL
CREATE PROCEDURE GetEmployee
AS
SELECT *
FROM Employee;
```

Execute whenever needed:

```
SQL
EXEC GetEmployee;
```

4. Advantages of Stored Procedures

1. Code Reusability

Write once, execute many times.

Example:

```
Create Procedure
    |
    |
Use anywhere
```

2. Better Performance

Stored procedures are usually faster because:

- SQL statements are precompiled
- Execution plan can be reused

3. Security

Users can execute procedures without directly accessing tables.

Example:

Employee cannot access:

Salary Table

but can run:

GetSalary Procedure

4. Reduced Network Traffic

Instead of sending many SQL statements:

Application sends:

```
EXEC Procedure_Name
```

Only one request is sent.

5. Easier Maintenance

Changing logic in one place updates all applications.

5. Basic Stored Procedure Syntax

SQL Server

```
SQL
CREATE PROCEDURE procedure_name
AS
BEGIN
SQL statements
END;
```

Example:

```
SQL
CREATE PROCEDURE GetStudents
AS
BEGIN
SELECT *
FROM Student;
END;
```

Execute:

```
SQL
EXEC GetStudents;
```

6. Stored Procedure with Parameters

Parameters allow passing values dynamically.

Syntax:

```
SQL
CREATE PROCEDURE procedure_name
@parameter datatype
AS
BEGIN
SQL statements
END;
```

Example:

Find employee by ID:

```
SQL
CREATE PROCEDURE GetEmployeeByID
@EmpID INT
AS
BEGIN
SELECT *
FROM Employee
WHERE ID=@EmpID;
END;
```

Execute:

```
SQL
EXEC GetEmployeeByID 101;
```

Output:

ID	Name
101	Rahul

7. Input Parameters

Input parameters send values into a procedure.

Example:

```
SQL
CREATE PROCEDURE AddEmployee
@Name VARCHAR(50),
@Salary INT
AS
BEGIN
```

```
INSERT INTO Employee
VALUES(@Name,@Salary);

END;
```

Execution:

```
SQL
EXEC AddEmployee
'Priya',
50000;
```

8. Output Parameters

Output parameters return values from procedures.

Example:

Find employee count:

```
SQL
CREATE PROCEDURE EmployeeCount
@Count INT OUTPUT
AS
BEGIN
SELECT @Count = COUNT(*)
FROM Employee;
END;
```

Execute:

```
SQL
DECLARE @Total INT;
EXEC EmployeeCount @Total OUTPUT;
PRINT @Total;
```

9. Stored Procedure Example: Banking System

Task:

Transfer money between accounts.

```
SQL
CREATE PROCEDURE TransferMoney
@FromAccount INT,
@ToAccount INT,
@Amount INT
AS
BEGIN
```

```
UPDATE Account
SET Balance = Balance - @Amount
WHERE Account_ID=@FromAccount;

UPDATE Account
SET Balance = Balance + @Amount
WHERE Account_ID=@ToAccount;

END;
```

Execution:

```
SQL
EXEC TransferMoney
101,
102,
5000;
```

10. Stored Procedure in Oracle

Oracle uses PL/SQL.

Syntax:

```
SQL
CREATE OR REPLACE PROCEDURE procedure_name
AS
BEGIN
SQL statements;
END;
/
```

Example:

```
SQL
CREATE OR REPLACE PROCEDURE GetStudents
AS
BEGIN
SELECT *
FROM Student;
END;
/
```

11. Stored Procedure vs Function

Stored Procedure	Function
May or may not return value	Must return a value
Can perform INSERT/UPDATE/DELETE	Usually used for calculations
Called using EXEC	Used inside SQL statements
Can have output parameters	Returns single value

Example:

Function:

```
SQL
SELECT CalculateTax(Salary)
FROM Employee;
```

Procedure:

```
SQL
EXEC UpdateSalary;
```

12. Stored Procedure vs View

Stored Procedure	View
Contains SQL logic	Contains SQL query
Can accept parameters	Cannot accept parameters
Can perform operations	Mostly used for retrieval
More powerful	Simpler

13. Stored Procedure Flow

```

Application
  |
  ↓
EXEC Procedure
  |
  ↓
Database Engine
  |
  ↓
Execute SQL Statements
  |
  ↓
Return Result

```

14. Error Handling in Stored Procedures

SQL Server example:

```
SQL
CREATE PROCEDURE TestProcedure
AS
BEGIN
BEGIN TRY

SELECT *
FROM Employee;

END TRY

BEGIN CATCH

PRINT 'Error Occurred';

END CATCH

END;
```

15. Real-World Uses

Banking

Procedures:

- Money transfer
 - Loan calculation
 - Account statement
-

E-Commerce

Procedures:

- Place order
 - Update inventory
 - Calculate discount
-

Hospital

Procedures:

- Patient registration
 - Billing generation
-

Interview Questions

Q1. What is a Stored Procedure?

Answer:

A stored procedure is a precompiled collection of SQL statements stored in a database and executed when required.

Q2. Advantages of Stored Procedures?

- Performance improvement
 - Security
 - Code reuse
 - Reduced network traffic
 - Easy maintenance
-

Q3. Difference between Stored Procedure and Function?

A procedure may not return a value, while a function always returns a value.

Q4. Can Stored Procedures accept parameters?

Yes.

Types:

- Input parameters
 - Output parameters
-

Q5. Where are Stored Procedures stored?

They are stored inside the database server.

Quick Revision Notes

Stored Procedure:

Reusable SQL program stored in database

Benefits:

- ✓ Performance
- ✓ Security
- ✓ Reusability
- ✓ Less network traffic

Types of Parameters:

Input
Output

Commands:

CREATE PROCEDURE

EXEC procedure_name

Next Topic:

SQL Triggers

We will cover:

- What is a Trigger?
- Types of Triggers
- BEFORE and AFTER triggers
- INSERT/UPDATE/DELETE triggers
- Real-world examples
- Stored Procedure vs Trigger
- Interview questions

Proceeding step-by-step.

Ok next

Module 4: Database Systems

Topic 15: SQL Triggers

1. What is a Trigger?

A **Trigger** is a special type of stored program that **automatically executes when a specific event occurs in a database**.

In simple words:

A Trigger is an automatic action performed by the database when INSERT, UPDATE, or DELETE operations happen.

2. Real-World Example

Banking System

When a customer withdraws money:

Operation:

```
SQL
UPDATE Account
SET Balance = Balance - 5000;
```

Automatically:

- Record transaction history
- Update audit table
- Send notification

A trigger can perform these tasks automatically.

3. Why Do We Need Triggers?

Triggers are used for:

- Maintaining data integrity
- Auditing changes
- Automatic calculations
- Enforcing business rules
- Logging activities

4. Trigger Working

Normal flow:

```
User
|
↓
SQL Command
|
↓
Database Table
```

With Trigger:

```
User
|
↓
SQL Command
|
↓
Trigger Executes Automatically
|
```

↓
Table Updated

5. Trigger Events

Triggers execute when these operations occur:

Event	Example
INSERT	Adding new record
UPDATE	Changing existing record
DELETE	Removing record

6. Types of Triggers

Triggers are classified based on:

1. Timing
2. Event

A) Based on Timing

1. BEFORE Trigger

Executes before the operation.

Example:

Before inserting employee data:

- Check salary
- Validate age

Flow:

```
BEFORE INSERT
Trigger
↓
Insert Data
```

2. AFTER Trigger

Executes after the operation completes.

Example:

After inserting employee:

- Create audit record

Flow:

```
Insert Data
↓
Trigger
```

B) Based on Event

1. INSERT Trigger

Runs when new data is inserted.

Example:

```
SQL
INSERT INTO Employee
VALUES(101, 'Rahul');
```

Trigger executes.

2. UPDATE Trigger

Runs when data changes.

Example:

```
SQL
UPDATE Employee
SET Salary=60000;
```

3. DELETE Trigger

Runs when records are deleted.

Example:

```
SQL
DELETE FROM Employee
WHERE ID=101;
```

7. SQL Server Trigger Syntax

```
SQL
CREATE TRIGGER trigger_name
ON table_name
AFTER INSERT
```

```
AS
BEGIN
SQL statements
END;
```

8. Example: Employee Audit Trigger

Suppose we have:

Employee Table

ID	Name	Salary
101	Rahul	50000

Audit Table

ID	Action	Date
----	--------	------

Create trigger:

```
SQL
CREATE TRIGGER Employee_Insert_Audit
ON Employee
AFTER INSERT
AS
BEGIN
INSERT INTO Audit
VALUES
('Employee Added',
GETDATE());
END;
```

Now whenever:

```
SQL
INSERT INTO Employee
VALUES(102, 'Priya', 60000);
```

Automatically:

Audit table gets:

Action	Date
Employee Added	Current Date

9. UPDATE Trigger Example

Track salary changes.

```
SQL
CREATE TRIGGER Salary_Update
ON Employee
AFTER UPDATE
AS
BEGIN
INSERT INTO Salary_Log
VALUES('Salary Changed',
GETDATE());
END;
```

When:

```
SQL
UPDATE Employee
SET Salary=70000
WHERE ID=101;
```

Trigger runs automatically.

10. DELETE Trigger Example

Backup deleted records.

```
SQL
CREATE TRIGGER Employee_Delete
ON Employee
AFTER DELETE
AS
BEGIN
INSERT INTO Deleted_Employee
SELECT *
FROM deleted;
END;
```

When employee is deleted:

Data is copied to backup table.

11. INSERTED and DELETED Tables

SQL Server provides temporary tables inside triggers.

INSERTED Table

Contains new values.

Example:

Before:

```
Salary = 50000
```

Update:

```
SQL
UPDATE Employee
SET Salary=60000;
```

Inserted table:

```
Salary = 60000
```

DELETED Table

Contains old values.

Example:

Before update:

```
Salary = 50000
```

Deleted table:

```
Salary = 50000
```

12. Trigger Example: Salary Validation

Rule:

Salary cannot be less than 10000.

```
SQL
CREATE TRIGGER Check_Salary
ON Employee
```

```

INSTEAD OF INSERT
AS
BEGIN
IF EXISTS
(
SELECT *
FROM inserted
WHERE Salary < 10000
)
BEGIN
PRINT 'Invalid Salary';
END
ELSE
BEGIN
INSERT INTO Employee
SELECT *
FROM inserted;
END
END;

```

13. INSTEAD OF Trigger

Meaning:

Runs instead of the original operation.

Example:

User executes:

```

SQL
DELETE FROM Employee;

```

Instead of deleting:

Trigger executes:

Move data to archive

14. Trigger vs Stored Procedure

Trigger	Stored Procedure
Runs automatically	Called manually
Associated with table	Independent
Cannot accept parameters normally	Can accept parameters
Used for automatic actions	Used for business logic

15. Advantages of Triggers

Advantage	Explanation
Automation	Runs automatically
Security	Controls changes
Auditing	Tracks modifications
Consistency	Maintains rules

16. Disadvantages of Triggers

Disadvantage	Explanation
Hidden execution	Runs without user knowing
Debugging difficulty	Hard to trace
Performance impact	Too many triggers slow database
Complexity	Database logic becomes complicated

17. Real-World Applications

Banking

Triggers:

- Record transactions
- Detect suspicious activity

E-Commerce

Triggers:

- Update inventory after order
- Calculate stock

Healthcare

Triggers:

- Maintain patient history
 - Audit medical records
-

Interview Questions

Q1. What is a Trigger?

Answer:

A trigger is a database object that automatically executes when INSERT, UPDATE, or DELETE operations occur.

Q2. Difference between Trigger and Stored Procedure?

Trigger:

- Automatic execution

Stored Procedure:

- Manual execution
-

Q3. What are the types of triggers?

Based on timing:

- BEFORE
- AFTER
- INSTEAD OF

Based on event:

- INSERT
 - UPDATE
 - DELETE
-

Q4. What are INSERTED and DELETED tables?

Temporary tables used in SQL Server triggers to store new and old values.

Q5. Can a trigger call a stored procedure?

Yes, a trigger can execute stored procedures.

Quick Revision Notes

TRIGGER:

Automatic SQL program

Runs on:

INSERT
UPDATE
DELETE

Types:

BEFORE
AFTER
INSTEAD OF

Uses:

- ✓ Auditing
- ✓ Validation
- ✓ Automatic updates

Trigger ≠ Procedure

Trigger → Automatic
Procedure → Manual

Next Topic:

Database Indexing

We will cover:

- What is an Index?
- How indexes work internally
- Clustered Index
- Non-Clustered Index
- B-Tree structure
- Advantages and disadvantages
- SQL examples
- Interview questions

Proceeding step-by-step.

Ok next

Module 4: Database Systems

Topic 16: Database Indexing

1. What is an Index?

An **Index** is a database object that improves the **speed of data retrieval operations**.

It works like the index of a book.

Example:

Without Book Index

Finding "Database Systems" requires reading every page.

With Book Index

Go directly to the required page number.

Similarly:

Without database index:

Search → Scan entire table

With index:

Search → Find location quickly → Fetch data

2. Why Do We Need Indexing?

Suppose we have a Student table:

ID	Name	Branch
1	Rahul	CSE
2	Priya	ECE
3	Amit	IT
4	John	CSE

Query:

```
SQL
SELECT *
FROM Student
WHERE ID=3;
```

Without index:

Database checks:

Row 1 → No
Row 2 → No
Row 3 → Yes

For millions of records, this is slow.

With index:

Index → Directly locate ID=3

Much faster.

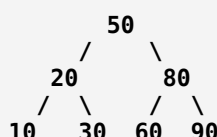
3. How Does an Index Work?

Most databases use a structure called:

B-Tree (Balanced Tree)

Example:

Index on Student_ID:



Searching:

Find 60:



Only a few comparisons are required.

4. Creating an Index

Syntax:

```
SQL
CREATE INDEX index_name
ON table_name(column_name);
```

Example:

Student table:

```
SQL
CREATE INDEX idx_student_id
ON Student(ID);
```

Now query:


```
SQL
SELECT *
FROM Student
WHERE ID=101;
```

will be faster.

5. Types of Indexes

Main types:

1. Clustered Index
2. Non-Clustered Index
3. Unique Index
4. Composite Index

1. Clustered Index

Definition:

A clustered index determines the **physical order of data in a table**.

The actual table data is stored according to the index.

Example:

Employee table:

Before:

ID	Name
3	Amit
1	Rahul
2	Priya

After clustered index on ID:

ID	Name
1	Rahul
2	Priya
3	Amit

Features:

- Only one clustered index per table
- Usually created on primary key
- Faster data retrieval

Example:

```
SQL
CREATE CLUSTERED INDEX
idx_employee_id
ON Employee(ID);
```

2. Non-Clustered Index

Definition:

A non-clustered index stores a separate structure containing:

- Indexed column
- Pointer to actual data

Structure:

```
Index
Name
|
Pointer
|
Actual Row
```

Example:

Create index:

```
SQL
CREATE NONCLUSTERED INDEX
idx_name
ON Employee(Name);
```

Search:

```
SQL
SELECT *
FROM Employee
WHERE Name='Rahul';
```

Index finds location quickly.

Features:

- Multiple non-clustered indexes allowed
- Does not change physical table order

Clustered vs Non-Clustered Index

Clustered	Non-Clustered
Changes physical order	Separate structure
One per table	Multiple allowed
Faster retrieval	Slightly slower
Stores actual data	Stores pointer to data

3. Unique Index

Ensures indexed column values are unique.

Example:

```
SQL
CREATE UNIQUE INDEX
idx_email
ON Employee(Email);
```

Invalid:

Email
a@gmail.com
a@gmail.com

4. Composite Index

Index created on multiple columns.

Example:

Table:

```
Student
ID
Name
Branch
```

Create index:

```
SQL
CREATE INDEX idx_student
ON Student(Name,Branch);
```

Useful query:

```
SQL
SELECT *
FROM Student
WHERE Name='Rahul'
AND Branch='CSE';
```

6. Primary Key and Index

When we create:

```
SQL
CREATE TABLE Student(
ID INT PRIMARY KEY,
Name VARCHAR(50)
);
```

Database automatically creates an index on ID.

7. Advantages of Indexing

Advantage	Explanation
Faster searching	Quick data retrieval
Improves query performance	Queries execute faster
Efficient sorting	ORDER BY becomes faster
Better joins	Faster table relationships

8. Disadvantages of Indexing

Disadvantage	Explanation
Uses extra storage	Index requires memory
Slower INSERT	Index must be updated
Slower UPDATE	Index values change
Maintenance required	Needs optimization

9. When Should We Create Indexes?

Good candidates:

Columns frequently used in:

WHERE

Example:

```
SQL
WHERE Employee_ID=101
```

JOIN

Example:

```
SQL
ON Student.Dept_ID=
Department.Dept_ID
```

ORDER BY

Example:

```
SQL
ORDER BY Salary
```

10. When Should We Avoid Indexes?

Avoid on:

- Small tables
- Frequently changing columns
- Columns with few unique values

Example:

Gender:

```
Male
Female
```

Index gives little benefit.

11. Removing an Index

Syntax:

```
SQL
```

```
DROP INDEX index_name;
```

Example:

```
SQL
DROP INDEX idx_student_id;
```

12. Index Example: E-Commerce

Product Table:

Product_ID	Name	Category	Price
1	Laptop	Electronics	50000
2	Phone	Electronics	20000

Query:

```
SQL
SELECT *
FROM Product
WHERE Category='Electronics';
```

Create index:

```
SQL
CREATE INDEX idx_category
ON Product(Category);
```

Search becomes faster.

13. Indexing and Query Optimization

Database optimizer decides:

Without index:

Full Table Scan

With index:

```
Index Seek
  |
  ↓
Retrieve Rows
```

Interview Questions

Q1. What is an Index?

Answer:

An index is a database structure that improves data retrieval speed by providing faster access paths to records.

Q2. Why are indexes used?

- Faster searching
- Faster joins
- Faster sorting

Q3. Difference between clustered and non-clustered index?

Clustered	Non-Clustered
Physical order changed	Separate lookup structure
One per table	Many possible

Q4. Does indexing slow INSERT operations?

Yes. Because the index also needs to be updated when data changes.

Q5. Which index is automatically created with Primary Key?

A clustered index (in many database systems, including SQL Server by default).

Quick Revision Notes

INDEX:

Purpose:

Increase query speed

Types:

1. Clustered

- Physical data order
- One per table

2. Non-Clustered

- Separate structure
- Multiple allowed

Benefits:

- ✓ Faster SELECT

Drawbacks:

- ✓ Extra storage
- ✓ Slower INSERT/UPDATE

Next Topic:

Database Transactions

We will cover:

- Transaction concepts
- ACID properties (deep dive)
- Transaction states
- Concurrency control
- Locks
- Deadlocks
- Isolation levels
- Real-world examples
- Interview questions

Proceeding step-by-step.

Ok next

Ok next

Module 4: Database Systems

Topic 17: Database Transactions (Deep Dive)

1. What is a Transaction?

A **transaction** is a sequence of database operations that are executed as a **single logical unit of work**.

In simple words:

A transaction is a group of SQL operations that either complete completely or fail completely.

2. Real-World Example: Bank Transfer

Transfer ₹10,000 from Account A to Account B.

Operations:

Step 1: Deduct money from A


```
SQL
UPDATE Account
SET Balance = Balance - 10000
WHERE Account_ID = 'A';
```

Step 2: Add money to B

```
SQL
UPDATE Account
SET Balance = Balance + 10000
WHERE Account_ID = 'B';
```

These two operations together form one transaction.

Successful Transaction

```
Account A: -10000
+
Account B: +10000
COMMIT
```

Money transferred successfully.

Failed Transaction

Suppose:

- Money deducted from A
- Server crashes before adding to B

Problem:

```
Account A: -10000
Account B: No change
```

Database uses **ROLLBACK** to undo the first operation.

3. Transaction Control Commands

Command	Purpose
COMMIT	Save changes permanently
ROLLBACK	Undo changes
SAVEPOINT	Create checkpoint

1. COMMIT

Makes transaction changes permanent.

Example:

```
SQL
UPDATE Employee
SET Salary=70000
WHERE ID=101;

COMMIT;
```

After COMMIT:

- Data is saved permanently.
- Cannot be undone.

2. ROLLBACK

Cancels changes before commit.

Example:

```
SQL
DELETE FROM Student
WHERE ID=101;

ROLLBACK;
```

Deleted record returns.

3. SAVEPOINT

Creates a temporary point inside a transaction.

Example:

```
SQL
UPDATE Account
SET Balance=9000
WHERE ID=101;

SAVEPOINT A;

UPDATE Account
SET Balance=8000
WHERE ID=101;

ROLLBACK TO A;
```

Final balance:

9000

4. ACID Properties of Transactions

Every reliable transaction follows ACID.

A - Atomicity

Definition:

Transaction is treated as one indivisible unit.

Either:

- All operations execute
- None execute

Example:

Money transfer:

```
Debit Account A
+
Credit Account B
```

If credit fails:

```
Undo Debit
```

C - Consistency

Definition:

Database must move from one valid state to another valid state.

Example:

Before transfer:

```
Total Balance = ₹1,00,000
```

After transfer:

```
Total Balance = ₹1,00,000
```

No money should be created or lost.

I - Isolation

Definition:

Multiple transactions executing at the same time should not affect each other.

Example:

Two users accessing the same bank account:

Transaction 1:

Withdraw ₹5000

Transaction 2:

Check Balance

Database isolates them.

D - Durability

Definition:

Once committed, changes are permanent.

Even if:

- Power failure
- System crash
- Restart

Data remains.

5. Transaction States

A transaction passes through different states:

```

Active
  |
  ↓
Partially Committed
  |
  ↓
Committed
  
```

Failure path:

```

Active
  |
  ↓
Failed
  |
  
```

↓
Rollback

1. Active State

Transaction is currently executing.

Example:

```
SQL
UPDATE Account;
```

2. Partially Committed

Last operation completed.

Waiting for permanent storage.

3. Committed

Transaction successfully completed.

4. Failed

Transaction cannot continue.

Reasons:

- System crash
 - Constraint violation
 - Network failure
-

5. Terminated

Transaction leaves the system.

6. Concurrency in Transactions

What is Concurrency?

Concurrency means multiple transactions executing at the same time.

Example:

Online banking:

User A → Withdraw money

User B → Check balance

User C → Deposit money

All happen together.

7. Problems in Concurrent Transactions

1. Lost Update Problem

Two transactions update the same data.

Example:

Initial balance:

₹10000

Transaction A:

Withdraw 3000

Transaction B:

Withdraw 2000

Both read:

₹10000

Final result may become incorrect.

2. Dirty Read

A transaction reads uncommitted data.

Example:

Transaction A:

Update salary = 90000

Not committed.

Transaction B:

Reads salary = 90000

Later:

Transaction A rollback.

Actual salary:

50000

Transaction B read wrong data.

3. Non-Repeatable Read

Same query gives different results.

Example:

Transaction A:

SELECT Balance

Result:

50000

Transaction B updates balance:

70000

Transaction A reads again:

70000

Same query, different output.

4. Phantom Read

A transaction sees new rows added by another transaction.

Example:

First query:

```
SQL
SELECT *
FROM Employee
WHERE Department='CSE';
```

Output:

10 employees

Another transaction inserts:

1 new CSE employee

Second query:

```
11 employees
```

New row appears.

8. Locks in Database

Locks prevent conflicts between transactions.

Types:

1. Shared Lock (S Lock)

Used for reading.

Multiple transactions can read.

Example:

```
Transaction A → Read
Transaction B → Read
```

Allowed

2. Exclusive Lock (X Lock)

Used for writing.

Only one transaction can modify data.

Example:

```
Transaction A → Update
```

```
Transaction B → Wait
```

9. Deadlock

Definition:

A deadlock occurs when two transactions wait for each other forever.

Example:

Transaction A:

```
Locks Table 1
Waits for Table 2
```

Transaction B:

Locks Table 2
Waits for Table 1

Both wait forever.

Deadlock Solution:

Database may:

- Detect deadlock
 - Rollback one transaction
 - Release locks
-

10. Transaction Isolation Levels

Isolation levels control how transactions interact.

Levels:

1. Read Uncommitted
 2. Read Committed
 3. Repeatable Read
 4. Serializable
-

1. Read Uncommitted

Lowest isolation.

Allows dirty reads.

Fast but less safe.

2. Read Committed

Prevents dirty reads.

Default in many databases.

3. Repeatable Read

Prevents:

- Dirty reads
- Non-repeatable reads

4. Serializable

Highest isolation.

Transactions execute as if sequentially.

Most secure but slower.

Isolation Level Comparison

Level	Dirty Read	Non-Repeatable	Phantom
Read Uncommitted	Possible	Possible	Possible
Read Committed	No	Possible	Possible
Repeatable Read	No	No	Possible
Serializable	No	No	No

11. Transaction Example: Online Shopping

Order placement:

```

1. Create Order
   ↓
2. Reduce Inventory
   ↓
3. Process Payment
   ↓
4. Confirm Order
  
```

If payment fails:

ROLLBACK

Undo:

- Order creation
- Inventory reduction

Interview Questions

Q1. What is a transaction?

A transaction is a collection of SQL operations treated as a single unit of work.

Q2. Explain ACID properties.

A:
Atomicity - All or nothing

C:
Consistency - Valid database state

I:
Isolation - Transactions don't interfere

D:
Durability - Permanent changes

Q3. What is a deadlock?

A situation where two transactions wait for each other indefinitely.

Q4. Difference between COMMIT and ROLLBACK?

COMMIT	ROLLBACK
Saves changes	Undo changes
Permanent	Temporary

Q5. What is concurrency control?

A mechanism to manage simultaneous transactions without data inconsistency.

Quick Revision Notes

TRANSACTION:

Group of SQL operations

Commands:

COMMIT

→ Save

ROLLBACK

→ Undo

SAVEPOINT

→ Checkpoint

ACID:

A → Atomicity

C → Consistency

I → Isolation

D → Durability

Problems:

Lost Update
Dirty Read
Non-repeatable Read
Phantom Read

Locks:

Shared Lock
Exclusive Lock

Deadlock:
Transactions waiting forever

Next Topic:

SQL Server Basics & Oracle Basics

We will cover:

- SQL Server architecture
- SSMS tool
- Databases and schemas
- Oracle architecture
- SQL Developer
- SQL Server vs Oracle
- Basic commands
- Practical setup

Proceeding step-by-step.

Ok next

Today 8:25 AM

Module 10: C#

Topic 13: Collections in C# (Detailed Explanation)

Collections are one of the **most important topics in C#**. In real-world applications, you rarely know in advance how many items you need to store. Collections provide **dynamic, efficient, and flexible** ways to manage groups of objects.

Examples:

- User accounts
- Shopping cart items
- Employee records

- API responses
- Database query results

1. What are Collections?

A **collection** is an object that stores and manages multiple elements.

Unlike arrays:

- Arrays have a **fixed size**.
- Collections can **grow and shrink dynamically**.

Example:

```
Array
Size = Fixed
[10][20][30]

Collection (List)
10
20
30
40
50
(Add or Remove Anytime)
```

2. Why Use Collections?

Suppose you're storing students.

Using an array:

```
C#
string[] students = new string[3];
```

Only 3 students can be stored.

Using a **List**:

```
C#
List<string> students = new List<string>();

students.Add("John");
students.Add("Alice");
students.Add("David");
students.Add("Emma");
```

No size limit needs to be specified.

3. Generic Collections

Most collections in C# are **generic**.

Example:

```
C#
List<int>
Dictionary<string, int>
Stack<string>
Queue<double>
```

Advantages:

- Type safety
- Better performance
- No boxing/unboxing
- Compile-time error checking

4. List<T>

The most commonly used collection.

A List<T> is a **dynamic array**.

Creating a List

```
C#
List<string> names = new List<string>();
```

Adding Items

```
C#
names.Add("John");
names.Add("Alice");
names.Add("David");
```

Memory:

```
Index
0 → John
1 → Alice
2 → David
```

Accessing Elements

```
C#
Console.WriteLine(names[1]);
```

Output:

```
Alice
```

Removing Items

```
C#
names.Remove("Alice");
```

Result:

```
John
David
```

Useful List Methods

```
C#
names.Add("Bob");
names.Remove("John");
names.Contains("Bob");
names.Clear();
names.Count;
```

Method	Purpose
Add()	Add item
Remove()	Remove item
Contains()	Check if exists
Clear()	Remove all items
Count	Number of items

5. Dictionary<TKey, TValue>

A **Dictionary** stores **key-value pairs**.

Example:

```
C#
Dictionary<int, string> students =
    new Dictionary<int, string>();
```

Adding values:

```
C#
students.Add(101, "John");
students.Add(102, "Alice");
students.Add(103, "David");
```

Memory:

```
101 → John
102 → Alice
```

103 → David

Access Value

```
C#
Console.WriteLine(students[102]);
```

Output:

Alice

Check Key

```
C#
students.ContainsKey(101);
```

Remove

```
C#
students.Remove(102);
```

Iterate

```
C#
foreach(var item in students)
{
    Console.WriteLine($"{item.Key} {item.Value}");
}
```

Real-World Use

- Employee ID → Employee
- Product ID → Product
- Username → User Profile

6. HashSet<T>

A `HashSet<T>` stores **unique values only**.

Example:

```
C#
HashSet<int> numbers = new HashSet<int>();

numbers.Add(10);
numbers.Add(20);
numbers.Add(20);
numbers.Add(30);
```

Result:


```
10
20
30
```

Duplicate 20 is ignored.

Common Uses

- Remove duplicates
- Store unique usernames
- Tags
- Permissions

7. Queue<T>

A Queue follows **FIFO (First In, First Out)**.

Example:

```
Front
John
Alice
David
Rear
```

The first person added leaves first.

Operations

```
C#
Queue<string> queue = new Queue<string>();
queue.Enqueue("John");
queue.Enqueue("Alice");
queue.Enqueue("David");
```

Remove:

```
C#
queue.Dequeue();
```

Output:

```
John Removed
```

Peek:

```
C#
queue.Peek();
```

Output:

```
Alice
```

Real-World Example

```
Printer Queue
Print Job1
Print Job2
Print Job3
```

8. Stack<T>

A Stack follows **LIFO (Last In, First Out)**.

Example:

```
Top
Book3
Book2
Book1
```

The last book placed is removed first.

Operations

```
C#
Stack<int> stack = new Stack<int>();
stack.Push(10);
stack.Push(20);
stack.Push(30);
```

Remove:

```
C#
stack.Pop();
```

Output:

```
30
```

Peek:

```
C#
stack.Peek();
```

Output:

```
20
```

Real-World Example

- Browser Back button
- Undo feature
- Function call stack

9. Choosing the Right Collection

Collection	Best Used For
Array	Fixed-size data
List<T>	Dynamic ordered list
Dictionary<TKey,TValue>	Fast lookup by key
HashSet<T>	Unique values
Queue<T>	FIFO processing
Stack<T>	LIFO processing

10. Time Complexity (Important for Interviews)

Operation	List	Dictionary	HashSet	Queue	Stack
Access by Index	O(1)	—	—	—	—
Search	O(n)	O(1) average	O(1) average	O(n)	O(n)
Insert/Add	O(1) amortized	O(1) average	O(1) average	O(1)	O(1)
Remove	O(n)	O(1) average	O(1) average	O(1)	O(1)

Note: Dictionary and HashSet operations are **O(1) on average** but can degrade in rare cases due to hash collisions.

11. Generic vs Non-Generic Collections

Generic	Non-Generic
List<int>	ArrayList

Generic	Non-Generic
Type-safe	Stores object
Better performance	Requires boxing/unboxing for value types
Preferred in modern C#	Mainly kept for compatibility

Always prefer **generic collections** in new applications.

12. Real-World Examples

Shopping Cart

```
C#
List<Product> cart = new List<Product>();
```

Employee Directory

```
C#
Dictionary<int, Employee> employees;
```

Browser History

```
C#
Stack<string> history;
```

Print Queue

```
C#
Queue<PrintJob> jobs;
```

Unique Email Addresses

```
C#
HashSet<string> emails;
```

13. Common Mistakes

✗ Using an Array When Size Changes Frequently

Instead of:

```
C#
string[] names = new string[100];
```

Use:

```
C#
List<string> names = new List<string>();
```

✗ Using Dictionary Without Checking Key

Wrong:

```
C#
Console.WriteLine(dict[500]);
```

If key **500** doesn't exist, it throws a **KeyNotFoundException**.

Better:

```
C#
if(dict.ContainsKey(500))
{
    Console.WriteLine(dict[500]);
}
```

Or use `TryGetValue()`:

```
C#
if(dict.TryGetValue(500, out string value))
{
    Console.WriteLine(value);
}
```

✗ Expecting HashSet to Preserve Order

A `HashSet<T>` is designed for uniqueness, **not for maintaining insertion order**. Don't rely on its iteration order.

14. Best Practices

- ✓ Use `List<T>` instead of arrays when the size changes.
- ✓ Use `Dictionary<TKey, TValue>` for fast lookups.
- ✓ Use `HashSet<T>` to eliminate duplicates.
- ✓ Use `Queue<T>` for FIFO scenarios.
- ✓ Use `Stack<T>` for LIFO scenarios.

15. Interview Questions

Q1. Difference between Array and List?

Array	List
Fixed size	Dynamic size
Faster for fixed-size scenarios	Flexible and easier to manage

Q2. Difference between Dictionary and List?

Dictionary	List
Access by key	Access by index
Fast average lookup	Sequential search for values

Q3. What is FIFO?

First In, First Out. Used by `Queue<T>`.

Q4. What is LIFO?

Last In, First Out. Used by `Stack<T>`.

Q5. Why use HashSet?

To store **unique values** efficiently and avoid duplicates.

Complete Mental Model

```

Collections
├── Array
├── List<T>
├── Dictionary<TKey, TValue>
├── HashSet<T>
├── Queue<T>
└── Stack<T>
  
```

Quick Revision

```

Collections
↓
List<T>
↓
Dictionary
  
```

↓
HashSet
 ↓
Queue (FIFO)
 ↓
Stack (LIFO)
 ↓
Generic Collections
 ↓
Time Complexity
 ↓
Real-world Uses

Next Topic:

Module 10 – Topic 14: LINQ (Language Integrated Query) – Complete Detailed Guide

We'll cover:

- What is LINQ?
- Why LINQ is needed
- Query syntax vs Method syntax
- **Where()**
- **Select()**
- **OrderBy()**
- **GroupBy()**
- **First(), FirstOrDefault()**
- **Single(), Any(), All()**
- Aggregation (**Count, Sum, Average, Max, Min**)
- LINQ with Collections and Entity Framework
- Deferred execution
- Real-world examples
- Interview questions

LINQ is one of the most powerful features of C# and is used extensively in **ASP.NET Core, Entity Framework Core**, and **enterprise applications**.

Ok next

Module 10: C#

Topic 14: LINQ (Language Integrated Query) – Detailed Explanation

LINQ (Language Integrated Query) is one of the most powerful features in C#. It allows you to **query, filter, sort, group, and transform data** using a consistent syntax.

LINQ works with:

- Arrays
- Lists
- Dictionaries
- XML
- Databases (Entity Framework Core)
- JSON (after deserialization)

Interview Importance: LINQ is asked in almost every C#/.NET interview and is heavily used in ASP.NET Core and EF Core.

1. What is LINQ?

LINQ stands for **Language Integrated Query**.

It allows you to write SQL-like queries directly in C#.

Without LINQ:

```
C#
List<int> numbers = new List<int> { 10, 20, 30, 40, 50 };

foreach (int n in numbers)
{
    if (n > 25)
        Console.WriteLine(n);
}
```

With LINQ:

```
C#
var result = numbers.Where(n => n > 25);

foreach (var n in result)
{
    Console.WriteLine(n);
}
```

Output:

```
30
40
50
```


LINQ makes code shorter, cleaner, and easier to read.

2. Why Do We Need LINQ?

Imagine a list of employees:

John	50000
Alice	70000
David	40000
Emma	80000

You may want to:

- Find employees with salary > 50,000
- Sort employees by salary
- Group employees by department
- Count employees

LINQ provides simple methods to do all these tasks.

3. Two Ways to Write LINQ

A. Query Syntax

Looks similar to SQL.

```
C#
var result =
    from n in numbers
    where n > 20
    select n;
```

B. Method Syntax (Most Common)

```
C#
var result =
    numbers.Where(n => n > 20);
```

Both produce the same result.

In modern C# projects, **method syntax** is used more frequently.

4. Where() – Filtering Data

Used to filter elements.

Example:

```
C#
List<int> numbers =
    new List<int> {10,20,30,40,50};

var result =
    numbers.Where(n => n > 25);
```

Output:

```
30
40
50
```

Real-World Example

```
C#
var adults =
    students.Where(s => s.Age >= 18);
```

5. Select() – Projecting Data

Used to select or transform data.

Example:

```
C#
var squares =
    numbers.Select(n => n * n);
```

Output:

```
100
400
900
1600
2500
```

Real-World Example

Employee object:

```
Name
Salary
Department
```

Need only names:

```
C#
```

```
var names =
    employees.Select(e => e.Name);
```

6. OrderBy() and OrderByDescending()

Sort ascending:

```
C#
var sorted =
    numbers.OrderBy(n => n);
```

Output:

```
10
20
30
40
50
```

Descending:

```
C#
var sorted =
    numbers.OrderByDescending(n => n);
```

Output:

```
50
40
30
20
10
```

7. First() and FirstOrDefault()

First()

Returns the first element.

```
C#
var value =
    numbers.First();
```

Output:

```
10
```

If the collection is empty, it throws an exception.

FirstOrDefault()

```
C#
var value =
    numbers.FirstOrDefault();
```

If empty:

```
0
```

For reference types, it returns `null`.

Use `FirstOrDefault()` when an empty collection is possible.

8. Single() and SingleOrDefault()

Suppose Employee ID is unique.

```
C#
var employee =
    employees.Single(e => e.Id == 10);
```

Rules:

- Exactly one matching element → returns it.
- No matching element → exception.
- More than one matching element → exception.

`SingleOrDefault()` returns the default value if no match exists, but still throws if multiple matches exist.

9. Any() and All()

Any()

Checks whether at least one element satisfies a condition.

```
C#
bool exists =
    numbers.Any(n => n > 40);
```

Output:

```
True
```

All()

Checks whether every element satisfies a condition.

```
C#  
bool result =  
    numbers.All(n => n > 5);
```

Output:

```
True
```

10. Aggregation Functions

Count()

```
C#  
numbers.Count();
```

Output:

```
5
```

Sum()

```
C#  
numbers.Sum();
```

Output:

```
150
```

Average()

```
C#  
numbers.Average();
```

Output:

```
30
```

Max()

```
C#  
numbers.Max();
```

Output:

```
50
```

Min()

```
C#
numbers.Min();
```

Output:

```
10
```

11. GroupBy()

Groups data by a key.

Example:

```
C#
var groups =
    employees.GroupBy(e => e.Department);
```

Visualization:

```
IT
↓
John
Alice
-----
HR
↓
David
Emma
```

Useful for reports and analytics.

12. Chaining LINQ Methods

You can combine multiple operations.

```
C#
var result =
    numbers
        .Where(n => n > 20)
        .OrderByDescending(n => n)
        .Select(n => n);
```

Execution:

```
Numbers
↓
```

Where

↓

OrderBy

↓

Select

↓

Result

13. Deferred Execution (Important)

LINQ queries are **deferred** by default.

Example:

```
C#
var result =
    numbers.Where(n => n > 20);
```

No filtering happens yet.

The query executes only when you iterate over it:

```
C#
foreach(var n in result)
{
    Console.WriteLine(n);
}
```

Or when you call methods like:

```
C#
ToList()
ToArray()
Count()
First()
```

This improves efficiency because unnecessary work is avoided.

14. LINQ with Entity Framework Core

Example:

```
C#
var students =
    context.Students
        .Where(s => s.Age > 18)
        .OrderBy(s => s.Name)
        .ToList();
```

Entity Framework translates this LINQ query into SQL.

Equivalent SQL:

```
SQL
SELECT *
FROM Students
WHERE Age > 18
ORDER BY Name;
```

This is one of the biggest reasons LINQ is so powerful.

15. Real-World Examples

Shopping Website

```
C#
var expensiveProducts =
    products.Where(p => p.Price > 5000);
```

Student Portal

```
C#
var toppers =
    students.OrderByDescending(s => s.Marks);
```

Banking

```
C#
var total =
    accounts.Sum(a => a.Balance);
```

Employee Management

```
C#
var names =
    employees.Select(e => e.Name);
```

16. Common Mistakes

✗ Calling First() on an Empty Collection

```
C#
numbers.First();
```


Throws an exception if the collection is empty.

Prefer:

```
C#
numbers.FirstOrDefault();
```

✗ Forgetting Deferred Execution

```
C#
var query =
    numbers.Where(n => n > 20);
```

The query has **not executed yet**. It runs only when enumerated or materialized.

✗ Using Single() When Multiple Matches Are Possible

If duplicates may exist, use `FirstOrDefault()` or another appropriate method instead of `Single()`.

17. Best Practices

- ✓ Prefer method syntax in modern C#.
- ✓ Use `FirstOrDefault()` when data may be missing.
- ✓ Materialize results with `ToList()` if you'll iterate multiple times or need a snapshot.
- ✓ Keep LINQ expressions simple and readable.

18. Interview Questions

Q1. What is LINQ?

LINQ is a C# feature that provides a unified way to query and manipulate data from different sources.

Q2. Difference between Query Syntax and Method Syntax?

Query Syntax	Method Syntax
SQL-like	Uses extension methods and lambda expressions
Less common in modern projects	Most commonly used

Q3. Difference between First() and FirstOrDefault()?

First()	FirstOrDefault()
Throws if no element exists	Returns the default value if no element exists

Q4. Difference between Any() and All()?

Any()	All()
Checks if at least one element matches	Checks if every element matches

Q5. What is Deferred Execution?

LINQ delays executing a query until its results are actually needed (for example, during enumeration or after calling methods like `ToList()`).

Complete Mental Model



Quick Revision

Ok next

Module 10: C#

Topic 15: Delegates (Detailed Explanation)

Delegates are one of the most powerful features in C#. They are widely used in **events, LINQ, ASP.NET Core, Windows Forms, WPF, asynchronous programming, and callbacks**.

Interview Importance: Delegates are one of the most frequently asked C# interview topics.

1. What is a Delegate?

A **delegate** is a **type-safe function pointer**.

It stores a **reference to one or more methods** and allows you to call those methods indirectly.

Real-Life Example

Think of a TV remote.

```
Remote Button
↓
Calls TV Function
↓
Volume Up
```

The remote doesn't contain the TV logic; it only points to the function.

Similarly:

```
Delegate
↓
Points to Method
↓
Executes Method
```

2. Why Do We Need Delegates?

Normally, you call methods directly.

```
C#
Display();
```

With a delegate:

```
C#
delegateVariable();
```

The delegate decides **which method** to call at runtime.

This provides flexibility and loose coupling.

3. Declaring a Delegate

Syntax

```
C#
delegate returnType DelegateName(parameters);
```

Example:

```
C#
delegate void PrintDelegate();
```

This delegate can point to any method that:

- Returns **void**
- Takes no parameters

4. Using a Delegate

```
C#
using System;

delegate void PrintDelegate();

class Program
{
    static void Display()
    {
        Console.WriteLine("Hello Delegates");
    }

    static void Main()
    {
        PrintDelegate pd = Display;

        pd();
    }
}
```

Output:

```
Hello Delegates
```

Flow:

```
Delegate
↓
Display()
↓
```

Console Output

5. Delegate with Parameters

```
C#
delegate int AddDelegate(int a, int b);

class Program
{
    static int Add(int x, int y)
    {
        return x + y;
    }

    static void Main()
    {
        AddDelegate ad = Add;

        Console.WriteLine(ad(10,20));
    }
}
```

Output:

```
30
```

6. Multicast Delegates

A delegate can reference **multiple methods**.

```
C#
delegate void PrintDelegate();

class Program
{
    static void Hello()
    {
        Console.WriteLine("Hello");
    }

    static void Welcome()
    {
        Console.WriteLine("Welcome");
    }

    static void Main()
    {
        PrintDelegate pd = Hello;

        pd += Welcome;

        pd();
    }
}
```

Output:

```
Hello
Welcome
```

Removing Methods

```
C#
pd -= Hello;
```

Now only `Welcome()` executes.

7. Built-in Delegates

Instead of creating your own delegate, C# provides built-in delegates.

A. Action

Represents a method that **returns nothing (void)**.

Example:

```
C#
Action message = () =>
{
    Console.WriteLine("Hello");
};

message();
```

Output:

```
Hello
```

Action with Parameters

```
C#
Action<string> greet = name =>
{
    Console.WriteLine($"Hello {name}");
};

greet("Kamesh");
```

Output:

```
Hello Kamesh
```

8. Func

Represents a method that **returns a value**.

Example:

```
C#
Func<int,int,int> add = (a,b) => a+b;

Console.WriteLine(add(5,3));
```

Output:

```
8
```

The last type parameter is always the **return type**.

9. Predicate

Represents a method that returns a **bool**.

Example:

```
C#
Predicate<int> isEven =
    x => x % 2 == 0;

Console.WriteLine(isEven(10));
```

Output:

```
True
```

Useful for filtering collections.

10. Lambda Expressions

Delegates are commonly used with **lambda expressions**.

Old way:

```
C#
delegate(int x)
{
    return x*x;
}
```

Modern way:

```
C#
x => x * x
```

Example:

```
C#
Func<int,int> square =
    x => x*x;
```

```
Console.WriteLine(square(5));
```

Output:

```
25
```

Lambda expressions are concise and heavily used with LINQ.

11. Delegates with LINQ

```
C#
List<int> numbers =
    new List<int>{1,2,3,4,5};

var even =
    numbers.Where(x => x % 2 == 0);
```

Here:

```
Where()
↓
Uses Delegate
↓
Lambda Expression
↓
Returns Matching Values
```

Output:

```
2
4
```

12. Real-World Example

Payment Gateway

```
Payment
↓
Choose Method
↓
Credit Card
UPI
Net Banking
```


Each payment method can be represented by a different method, and a delegate decides which one to execute.

Logging

```
Application
↓
Delegate
↓
Console Logger
↓
File Logger
↓
Database Logger
```

Different logging implementations can be plugged in without changing the calling code.

13. Common Mistakes

✗ Signature Mismatch

Delegate:

```
C#
delegate void Test();
```

Method:

```
C#
int Test()
```

✗ Error because the return types differ.

✗ Calling a Null Delegate

```
C#
PrintDelegate pd = null;

pd();
```

Throws a `NullReferenceException`.

Safer approach:

```
C#
```

```
pd?.Invoke();
```

`?.Invoke()` checks for `null` before invoking the delegate.

✗ Confusing Delegates with Methods

A delegate is **not a method**.

A delegate is an **object that references one or more methods**.

14. Best Practices

- ✓ Use **Action**, **Func**, and **Predicate** instead of creating custom delegates when they fit your needs.
- ✓ Use lambda expressions for concise code.
- ✓ Use `?.Invoke()` when invoking delegates that may be `null`.
- ✓ Keep delegate methods focused on a single responsibility.

15. Interview Questions

Q1. What is a delegate?

A delegate is a **type-safe function pointer** that can reference and invoke one or more methods with a matching signature.

Q2. What is a multicast delegate?

A delegate that references multiple methods. When invoked, it calls each method in the order they were added.

Q3. Difference between Action and Func?

Action	Func
Returns void	Returns a value
Can accept parameters	Can accept parameters and has a return value

Q4. What is a Predicate?

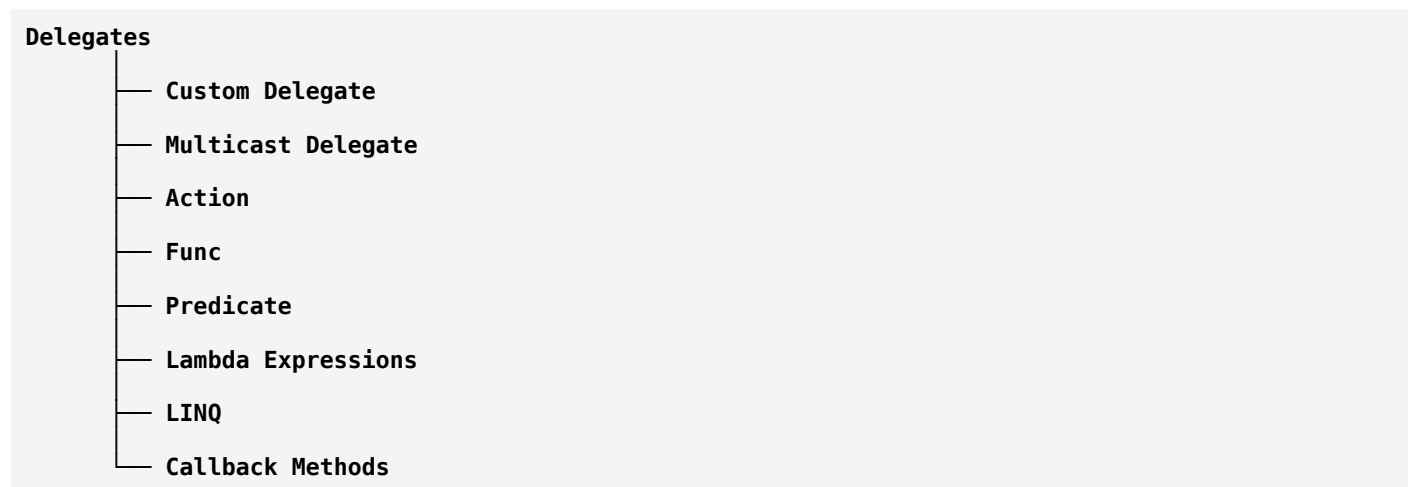
A delegate that takes one input parameter and returns a **bool**.

Q5. Why are delegates used?

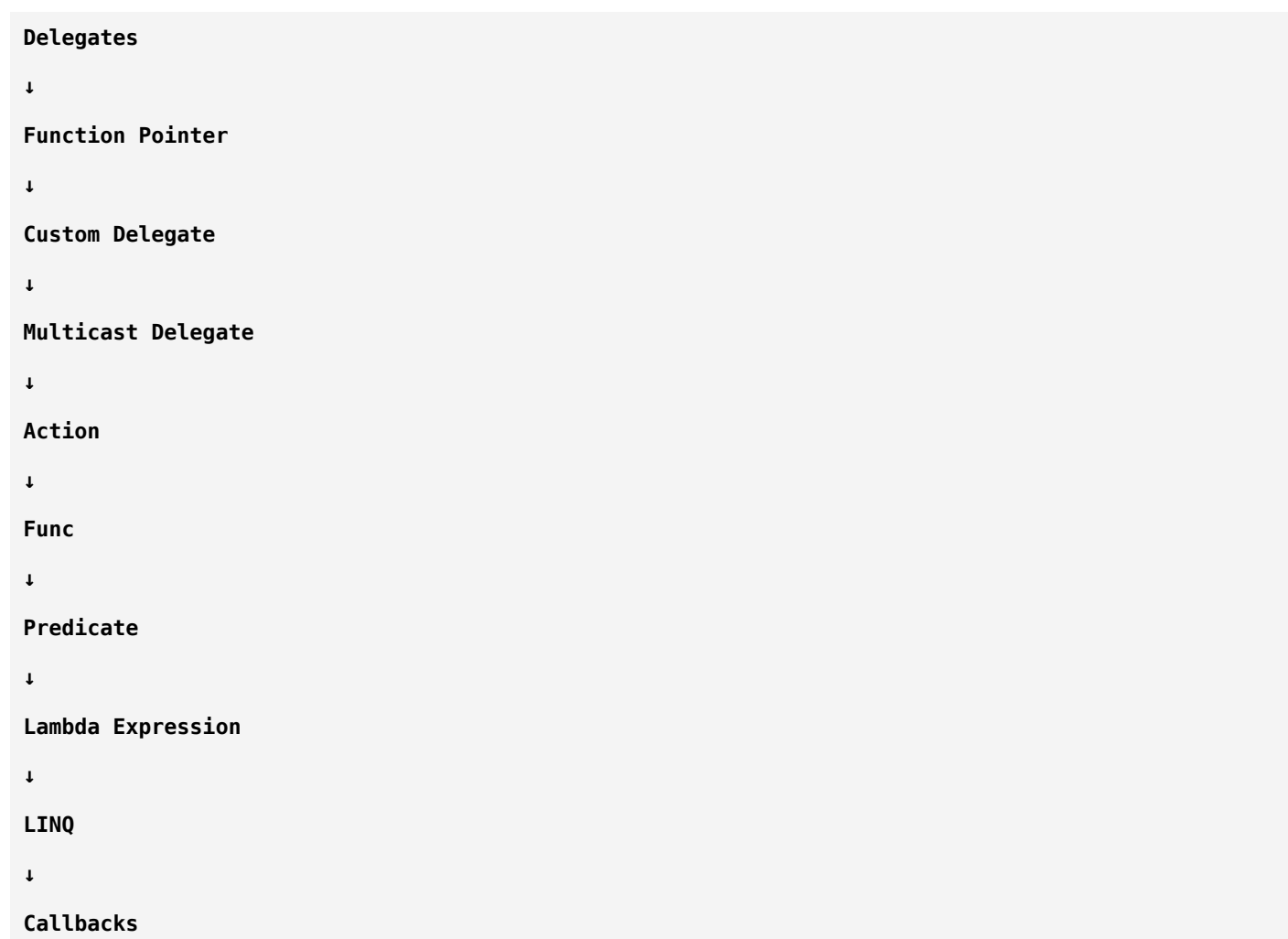
- Callbacks

- Event handling
- LINQ
- Asynchronous programming
- Loose coupling between components

Complete Mental Model



Quick Revision



Next Topic:

Module 10 – Topic 16: Events in C# (Complete Detailed Guide)

We'll cover:

- What are events?
- Publisher and subscriber model
- Creating and raising events
- Event handlers
- **event** keyword
- Built-in **EventHandler** delegate
- Custom event arguments
- Real-world examples (Button Click, Notifications)
- Best practices
- Interview questions

Events are built on top of delegates and are essential for **GUI applications, ASP.NET Core, and event-driven programming**.